

A MAJOR PROJECT REPORT ON

LEAFSIGHT: INTELLIGENT RICE DISEASE RECOGNITION USING VISION TRANSFORMERS AND GRU

An Explainable Deep Learning System for Rice Leaf Disease Recognition

Submitted in partial fulfillment of the requirements
for the award of the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

Student Name: [Enter Student Name]

Roll Number: [Enter Roll Number]

Department: [Enter Department]

College / Institution: [Enter College]

Academic Year: [Enter Academic Year]

Project Supervisor / Guide: [Enter Guide Name]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
2025 – 2026

CERTIFICATE OF APPROVAL

This is to certify that the project entitled "LEAFSIGHT: Intelligent Rice Disease Recognition using Vision Transformers and GRU" is a bona fide record of the major project work done by [Enter Student Name] (Roll No: [Enter Roll Number]) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, during the academic year [Enter Academic Year].

The work described in this report has been carried out under our supervision and guidance, and has not been submitted elsewhere for the award of any degree or diploma.

[Enter Guide Name]
Project Supervisor

Head of the Department
Dept. of CSE

External Examiner
Evaluation Committee

CANDIDATE'S DECLARATION

I hereby declare that the project work presented in this report entitled "LEAFSIGHT: Intelligent Rice Disease Recognition using Vision Transformers and GRU" is an authentic record of our own research and engineering work carried out under the supervision of [Enter Guide Name].

I confirm that all materials, datasets, models, code libraries, and external literature references utilized in this project have been fully acknowledged and cited in accordance with academic honesty standards. This report has not been submitted in whole or in part for any other academic degree, diploma, or certificate.

Submission Details:

Date: [Enter Date]

Place: [Enter City / College]

Signature of the Candidate

[Enter Student Name]

Roll No: [Enter Roll Number]

ACKNOWLEDGEMENTS

I express our deepest gratitude to our project guide, [Enter Guide Name], for their invaluable guidance, constructive technical feedback, and continuous encouragement throughout the conception, experimental design, and software implementation of the LEAFSIGHT project.

I would like to extend our sincere appreciation to the Head of the Department, [Enter Department], and the faculty members of the College for providing the computing infrastructure, laboratory facilities, and administrative support necessary for completing this work.

I also express our appreciation to the global open-source machine learning and agricultural research communities, specifically the developers of PyTorch, Hugging Face Transformers, FastAPI, and React, whose frameworks served as foundational tools for our research pipeline.

Author:

[Enter Student Name]

Department of Computer Science and Engineering

ABSTRACT

Rice (*Oryza sativa*) represents the primary dietary staple for over half the world's population. However, rice cultivation faces severe risks from foliar diseases, including Bacterial Blight (*Xanthomonas oryzae* pv. *oryzae*), Rice Blast (*Magnaporthe oryzae*), Brown Spot (*Bipolaris oryzae*), and Rice Tungro Disease. Traditional disease identification relies on manual visual inspection by agricultural extension specialists—a process that is labor-intensive, subjective, geographically constrained, and prone to misdiagnosis during early infection stages.

To address these limitations, this project presents LEAFSIGHT, an intelligent, explainable deep learning recognition system developed for automated rice leaf disease diagnosis. The proposed architecture employs a hybrid computer vision pipeline: a pretrained Vision Transformer (ViT-B/16) tokenizes 224×224 rice leaf images into 196 non-overlapping 16×16 spatial patches, yielding 768-dimensional token representations across 12 transformer encoder blocks. Unlike standard ViT classifiers that discard spatial patch structure in favor of a single global [CLS] token, LEAFSIGHT removes the [CLS] token and pipes the ordered sequence of 196 patch features into a Gated Recurrent Unit (GRU, hidden dimension 128) followed by a regularized dropout classifier ($p = 0.3$). The GRU aggregates spatial transition patterns and localized lesion boundaries across the foliar surface.

A multi-stage dataset audit was conducted across 5,932 raw images, resolving zero corruption, zero cross-class conflicts, and removing 1,096 duplicate groups (1,138 redundant copies) to construct a rigorously verified clean benchmark of 4,794 images. The dataset was partitioned into stratified 70% training (3,355 images), 15% validation (718 images), and 15% independent held-out test (721 images) splits. Training was executed via a two-stage progressive protocol: Stage A optimized the GRU and head with a frozen ViT backbone (AdamW, $\text{lr} = 1\text{e-}4$), reaching 98.75% validation accuracy; Stage B unfroze the top 2 ViT layers for controlled fine-tuning ($\text{lr} = 1\text{e-}5$), elevating validation accuracy to 99.72% (+0.97 percentage points).

On the independent 721-image held-out test set, LEAFSIGHT attained 99.86% classification accuracy, 99.83% macro precision, 99.88% macro recall, and 99.85% macro F1-score, correctly identifying 720 out of 721 test specimens (Brown Spot: 100%, Tungro: 100%, Blast: 99.65% F1, Bacterial Blight: 99.75% F1; single error misclassified at 78.78% confidence vs 99.61% mean correct confidence). For interpretability, a ViT attention rollout module computes recursive multi-head attention products across all 12 layers, generating 14×14 spatial saliency heatmaps overlaid on the original image. Finally, the pipeline was integrated into a production-grade full-stack web platform featuring a FastAPI backend and a React/Vite 'Vision Lab' interface equipped with an interactive 32-sample verified test console. LEAFSIGHT demonstrates high diagnostic fidelity, transparency, and computational accessibility for modern computational agriculture.

KEYWORDS: Rice Disease Recognition, Computer Vision, Vision Transformer, ViT-B/16, Gated Recurrent Unit (GRU), Deep Learning, Image Classification, Explainable AI, Attention Rollout, PyTorch, FastAPI, React

TABLE OF CONTENTS

Chapter 1: Introduction	1
1.1 Background & Context	1
1.2 Rice Disease Recognition	2
1.3 Problem Statement	2
1.4 Motivation & Agronomic Impact	3
1.5 Need for Automated Disease Recognition	3
1.6 Project Objectives	4
1.7 Scope of the Project	4
1.8 Summary of Major Contributions	5
1.9 Report Organization	5
Chapter 2: Problem Domain & Agronomic Pathology	6
2.1 Rice Plant Physiology & Vulnerability	6
2.2 Target Foliar Disease Classes	7
2.3 Challenges in Field Vision-Based Recognition	9
2.4 Role of Computational Intelligence in Plant Pathology	10
Chapter 3: Related Technology & Theoretical Foundations	11
3.1 Overview of Deep Learning in Plant Phenotyping	11
3.2 Convolutional Neural Networks vs. Vision Transformers	12
3.3 Vision Transformer (ViT-B/16) Architecture	13
3.4 Gated Recurrent Units (GRU) for Spatial Aggregation	15
3.5 Transfer Learning & Progressive Fine-Tuning	16
3.6 Explainable AI & Self-Attention Rollout	17
3.7 Software Frameworks & System Tooling	18
Chapter 4: Dataset Audit, Curation & Partitioning	19
4.1 Raw Dataset Acquisition	19
4.2 Multi-Stage Quality Audit & Deduplication	20
4.3 Clean Dataset Profile	21
4.4 Class Balance & Distribution Analysis	22
4.5 Stratified Partitioning Protocol	23
4.6 Visual Inspection of Verified Disease Classes	24
Chapter 5: Data Preprocessing & Augmentation Strategy	25
5.1 Channel Standardization & Normalization	25
5.2 Training Augmentation Pipeline	26
5.3 Deterministic Validation and Test Pipeline	27

5.4 Preprocessing Pipeline Visual Analysis	28
Chapter 6: Proposed Hybrid Architecture: ViT-B/16 + GRU	29
6.1 End-to-End Architectural Pipeline	29
6.2 Pretrained Vision Transformer Backbone	30
6.3 Patch Feature Extraction & [CLS] Token Removal	31
6.4 Gated Recurrent Unit (GRU) Sequence Aggregator	32
6.5 Regularization & Classification Head	33
6.6 Two-Stage Progressive Training Strategy	34
6.7 Computational Environment & Hardware Acceleration	36
Chapter 7: Experimental Evaluation & Benchmark Results	37
7.1 Evaluation Protocol on Independent 721-Image Test Set	37
7.2 Overall Performance Benchmark Analysis	38
7.3 Per-Class Performance Metrics	39
7.4 Confusion Matrix Analysis	40
7.5 Single Test Error In-Depth Dissection	41
7.6 Confidence & Softmax Calibration Analysis	42
7.7 Progressive Fine-Tuning Ablation Gain	43
Chapter 8: Explainable AI via Attention Rollout	44
8.1 The Black-Box Challenge in Agricultural AI	44
8.2 Self-Attention Rollout Mathematical Formulation	45
8.3 14 × 14 Spatial Patch Heatmap Interpolation	46
8.4 Visual Interpretation Across Disease Classes	47
8.5 Analysis of Misclassified Specimen Attention Map	48
8.6 Critical Scientific Distinction: Saliency vs. Causation	49
Chapter 9: Standalone Inference Pipeline	50
9.1 Python Inference Architecture	50
9.2 Image Validation & Format Standardization	51
9.3 Forward Pass Execution & Latency Profiling	52
9.4 Device Agnostic CUDA / CPU Failover	53
Chapter 10: Web Application & Vision Lab User Experience	54
10.1 Motivation for Decision-Support Interface	54
10.2 Frontend Architecture (React 18 + Vite)	55
10.3 Vision Lab Design System & Editorial Aesthetics	56
10.4 Core Interactive Components	57
10.5 Verified Test Console Workflow & Auto-Scroll	58
10.6 External Image Disclaimer & Scientific Honesty	59

Chapter 11: Full-Stack Software Architecture	60
11.1 Client-Server Communication Flow	60
11.2 FastAPI REST Backend Architecture	61
11.3 Reverse Proxy & Same-Origin Vite Configuration	62
Chapter 12: Codebase Organization & Repository Layout	63
12.1 Root Directory Hierarchy	63
12.2 Detailed File Responsibilities	64
Chapter 13: Modular Implementation Details	65
13.1 Dataset Audit & Verification Module	65
13.2 Preprocessing & Data Loading Module	66
13.3 ViTGRU Model Class Implementation	67
13.4 Two-Stage Training & Fine-Tuning Engines	68
13.5 Attention Rollout & Heatmap Synthesis Engine	69
13.6 FastAPI REST Endpoints & Handlers	70
13.7 React Vision Lab Component Hierarchy	71
Chapter 14: System Verification, Validation & Testing	72
14.1 Dataset Tests	72
14.2 Model Sanity Tests	72
14.3 Full Test-Set Benchmark Validation	73
14.4 REST API Integration & Health Testing	73
14.5 12-Sample Live Verification Test (100% Accuracy)	74
14.6 Production Build Verification	75
14.7 Testing Summary Matrix	75
Chapter 15: Limitations & Boundary Analysis	76
15.1 Domain Shift Sensitivity on Uncontrolled Photography	76
15.2 Lighting, Flash, and Glare Vulnerability	76
15.3 Single Organ Scope (Foliar Only)	77
15.4 Hardware Constraints & VRAM Boundaries	77
15.5 Interpretability Boundary: Saliency vs. Biological Causality	78
Chapter 16: Future Scope & Enhancement Pathways	79
16.1 Multimodal Sensor & Meteorological Fusion	79
16.2 Expansion to Panicle, Sheath, and Stem Pathologies	79
16.3 Edge ONNX / Mobile PWA Deployment	80
16.4 Weakly Supervised Lesion Localization	80
16.5 Active Learning Agronomist Feedback Loop	81
Chapter 17: Conclusion	82

17.1 Summary of Undertaking	82
17.2 Synthesis of Research & Engineering Results	83
17.3 Concluding Remarks	83
References	84
Appendices (A through H)	87

LIST OF FIGURES

Figure 1	Representative Rice Leaf Disease Classes in LEAFSIGHT Dataset	24
Figure 2	Dataset Distribution Across Splits (Clean Total: 4,794 Images)	24
Figure 3	Data Augmentation & Preprocessing Pipeline Visualization	28
Figure 4	End-to-End Deep Learning Architecture: ViT-B/16 + GRU + Rollout	30
Figure 5	Stage A Training and Validation Loss Convergence Curves	35
Figure 6	Stage A Training and Validation Accuracy Trajectory	35
Figure 7	Stage B Progressive Fine-Tuning Loss Convergence Curve	36
Figure 8	Stage B Progressive Fine-Tuning Accuracy Trajectory	36
Figure 9	Normalized Confusion Matrix on Independent 721-Image Test Set	40
Figure 10	Self-Attention Rollout Heatmap Visualization: Bacterial Blight	47
Figure 11	Self-Attention Rollout Heatmap Visualization: Rice Blast	47
Figure 12	Self-Attention Rollout Heatmap Visualization: Brown Spot	48
Figure 13	Self-Attention Rollout Heatmap Visualization: Rice Tungro	48
Figure 14	Attention Rollout Heatmap on Single Misclassified Test Specimen	49
Figure 15	LEAFSIGHT Vision Lab Web Application Interface & Test Console	50
Figure 16	Full-Stack Client-Server System Architecture Diagram	52

LIST OF TABLES

Table 1	Dataset Distribution and Stratified Split Statistics	22
Table 2	Deep Learning Model Architectural Specifications & Dimensions	31
Table 3	Stage A Training Hyperparameter Configuration	34
Table 4	Stage B Progressive Fine-Tuning Hyperparameter Configuration	35

Table 5	Summary Evaluation Metrics on 721-Sample Held-Out Test Set	38
Table 6	Detailed Per-Class Classification Report (Precision, Recall, F1)	39
Table 7	Numerical Confusion Matrix Breakdown (721 Test Samples)	40
Table 8	Prediction Confidence & Softmax Calibration Analysis	42
Table 9	Comprehensive System Verification & Testing Matrix	75
Table 10	Software Technologies, Libraries & Framework Versions	18

CHAPTER 1: INTRODUCTION

1.1 Background & Context

Agriculture serves as the backbone of rural livelihoods and global food security. Rice (*Oryza sativa*) is the preeminent staple grain for over 3.5 billion people worldwide, contributing upwards of 20% of global caloric intake. In developing economies across Asia and Africa, smallholder rice cultivation sustains millions of farming households. However, rice production faces intense challenges from changing climate conditions, resource constraints, and destructive phytopathogenic infections. Foliar diseases directly impair photosynthesis, disrupt nutrient transport, and precipitate catastrophic yield losses ranging from 20% to total crop failure if left unmitigated.

1.2 Rice Foliar Pathology & Agronomic Importance

The four primary foliar diseases investigated in this project represent major phytopathological threats to rice yields: Bacterial Blight, Rice Blast, Brown Spot, and Rice Tungro Disease. Bacterial Blight, caused by *Xanthomonas oryzae* pv. *oryzae*, causes systemic wilting and yellow-white necrotic lesions along the leaf margins. Rice Blast, caused by the ascomycete *Magnaporthe oryzae*, creates spindle-shaped lesions with grayish centers that can rapidly coalesce and destroy entire leaf blades. Brown Spot, caused by *Bipolaris oryzae*, produces circular to oval brown spots with prominent chlorotic halos, historically linked to severe food shortages such as the Great Bengal Famine of 1943. Rice Tungro Disease, caused by a viral complex of Rice Tungro Bacilliform Virus (RTBV) and Rice Tungro Spherical Virus (RTSV) transmitted by green leafhoppers (*Nephotettix virescens*), leads to severe stunting and vivid yellow-orange leaf discoloration.

1.3 Problem Statement

The conventional approach to rice disease diagnosis relies on manual visual examination by trained agronomists or plant pathologists. This paradigm exhibits critical limitations: (1) field diagnosis is highly labor-intensive and subjective, leading to substantial inter-observer variability; (2) agricultural extension officers are scarce in rural agrarian belts, resulting in delayed diagnosis; (3) early-stage lesions of distinct diseases frequently exhibit overlapping visual symptoms, inducing improper agrochemical application; and (4) conventional convolutional neural network (CNN) architectures struggle to capture long-range contextual relationships across foliar surfaces without excessive spatial pooling.

1.4 Motivation & Agronomic Impact

Automating foliar disease diagnosis through modern computer vision offers transformative potential for precision agriculture. Rapid, accurate diagnosis enables targeted intervention—applying specific bactericides, fungicides, or vector control measures precisely where and when required. This targeted response reduces input costs for farmers, minimizes environmental contamination from indiscriminate prophylactic chemical spraying, and preserves rice crop yields.

1.5 Need for Automated Disease Recognition

Recent advances in vision transformers have demonstrated superior feature extraction capabilities over traditional CNNs by modeling global self-attention across image patches. However, standard vision transformers rely on a single pooled token ([CLS]) that collapses spatial nuance. In foliar pathology, lesions exhibit directional expansion, spatial clustering, and edge boundaries. A hybrid framework that couples patch-level self-attention with sequential spatial modeling offers an effective pathway for robust agricultural phenotyping.

1.6 Project Objectives

- **Dataset Curation:** To construct a rigorously curated, verified, and deduplicated rice foliar disease benchmark dataset across four major disease classes.
- **Architectural Innovation:** To design and implement a novel hybrid deep learning architecture combining a pretrained Vision Transformer (ViT-B/16) with a Gated Recurrent Unit (GRU) sequence classifier.
- **Optimization Strategy:** To establish a two-stage progressive transfer learning protocol optimizing the sequence classifier while preserving and fine-tuning transformer representations on hardware-constrained GPU resources.
- **Explainable AI:** To implement self-attention rollout explainability, visualizing the spatial salience of patch tokens to justify model predictions transparently.
- **System Engineering:** To engineer a full-stack, responsive web application ('Vision Lab') integrating the trained model with FastAPI and React, providing an interactive verified test console.

1.7 Scope of the Project

The scope of LEAFSIGHT encompasses the end-to-end development of an image-based rice leaf disease recognition system. The project focuses on four verified classes: Bacterial Blight, Rice Blast, Brown Spot, and Tungro. The system accepts RGB leaf photographs, performs standardization and tokenization, generates calibrated disease probabilities, computes attention rollout heatmaps, and serves diagnoses through a web workstation. The project explicitly bounds its scope to leaf pathology under controlled and semi-controlled capture conditions, acknowledging domain shift boundaries.

1.8 Summary of Major Contributions

- **Deduplicated Dataset:** A clean 4,794-image dataset curated through automated multi-stage perceptual hash audits, removing 1,138 redundant duplicate copies with zero cross-class contamination.
- **Hybrid Architecture:** A hybrid ViT-B/16 + GRU network extracting 196 spatial patch tokens (16×16) and aggregating directional feature transitions without relying on the CLS token.
- **Benchmark Performance:** An empirical benchmark of 99.86% test accuracy, 99.85% Macro F1-score, and 720/721 correct predictions on a held-out test split of 721 images.
- **Model Interpretability:** Integrated recursive multi-head attention rollout explainability that visualizes 14×14 patch salience overlays on leaf specimens.
- **Deployable System:** A complete, modular web platform featuring interactive sample testing, real-time laser scanning animations, and domain-shift disclosures.

1.9 Report Organization

The remainder of this report is organized as follows: Chapter 2 examines the problem domain and foliar pathology. Chapter 3 surveys related technologies including ViTs and GRUs. Chapter 4 details dataset curation and audit procedures. Chapter 5 outlines preprocessing and augmentation. Chapter 6 presents the hybrid ViT+GRU model architecture and training protocol. Chapter 7 evaluates experimental test results. Chapter 8 details explainability via attention rollout. Chapter 9 covers standalone inference. Chapter 10 presents the web application. Chapter 11 documents software architecture. Chapter 12 details repository structure. Chapter 13 reviews modular implementation. Chapter 14 presents verification testing. Chapter 15 analyzes limitations. Chapter 16 outlines future scope, and Chapter 17 concludes the report.

CHAPTER 2: PROBLEM DOMAIN & AGRONOMIC PATHOLOGY

2.1 Rice Plant Physiology & Vulnerability

The rice plant (*Oryza sativa*) is a monocotyledonous annual grass characterized by slender, elongated leaves with parallel venation. The leaf blade (lamina) functions as the primary photosynthetic organ, responsible for synthesizing carbohydrates required for tillering, panicle emergence, and grain filling. Because rice is cultivated primarily in warm, humid tropical and subtropical environments (temperatures between 25°C and 35°C with relative humidity exceeding 80%), these microclimates concurrently create optimal breeding grounds for phytopathogenic fungi, bacteria, and insect vectors. The leaf epidermis, stomatal openings, and hydathodes act as natural ingress points for pathogens.

2.2 Target Foliar Disease Classes

LEAFSIGHT addresses the four most critical foliar diseases affecting global rice agriculture:

2.2.1 Bacterial Blight (*Xanthomonas oryzae* pv. *oryzae*)

Bacterial Blight is one of the most destructive bacterial diseases of rice. Pathogenic bacteria enter leaves through hydathodes at the leaf margin or through mechanical wounds caused by wind or human intervention. The disease manifests as water-soaked stripes that rapidly develop into undulating, yellowish-white necrotic bands extending longitudinally along the leaf veins. Under severe conditions, bacterial ooze forms on young lesions, and leaves dry out entirely (the 'kresek' symptom), causing yield losses up to 50%.

2.2.2 Rice Blast (*Magnaporthe oryzae*)

Rice Blast, caused by the filamentous fungus *Magnaporthe oryzae*, is capable of destroying entire fields within days under favorable conditions. On leaves, blast lesions initiate as small, water-soaked whitish or bluish specks that rapidly enlarge into diagnostic spindle-shaped (diamond-shaped or elliptical) lesions with pointed ends. The lesion centers become gray or whitish, surrounded by a dark reddish-brown border and frequently a diffuse chlorotic halo. Blast produces multiple airborne conidia, enabling rapid polycyclic epidemics.

2.2.3 Brown Spot (*Bipolaris oryzae*)

Brown Spot is a fungal disease typically associated with nutrient-deficient, poorly drained, or drought-stressed soils. Lesions on the foliar blade appear as circular to oval, reddish-brown to dark-brown spots. Fully developed spots display a light-brown to grayish center with a distinct dark-brown margin and yellow halo. Unlike blast lesions, brown spot lesions are generally smaller, more uniformly circular, and do not possess pointed spindle tips. In heavy infections, thousands of spots coalesce, causing premature leaf senescence.

2.2.4 Rice Tungro Disease (RTBV & RTSV)

Rice Tungro Disease is a devastating viral disease complex caused by the co-infection of Rice Tungro Bacilliform Virus (a pararetrovirus) and Rice Tungro Spherical Virus (an RNA virus). The virus is semi-persistently transmitted by leafhopper vectors, primarily the green leafhopper

Nephotettix virescens. Symptoms include marked plant stunting, reduced tillering, delayed flowering, and conspicuous yellow-to-orange discoloration of leaves that starts at the leaf tips and progresses downwards along the outer margins. Leaves become mottled, spiraled, and rust-colored.

2.3 Challenges in Field Vision-Based Recognition

- **Illumination Variability:** Natural sunlight, cloud cover, and flash reflection create severe shadow boundaries, overexposure, and glare on waxy rice leaf cuticles.
- **Cluttered Natural Backgrounds:** Leaves photographed in situ are surrounded by muddy paddy water, dense overlapping crop canopies, dry soil, and agricultural debris that confound naive background segmentation.
- **Orientation & Morphology:** Leaves exhibit natural curvature, twisting, drooping, and perspective distortion depending on wind and camera angle.
- **Symptom Confounding:** Early-stage bacterial blight streaks and young blast lesions can share similar pale yellow-brown coloration, complicating differentiation.
- **Progression Diversity:** Lesions evolve visually from water-soaked specks to necrotic scars, requiring models to recognize diverse disease stages within the same class.
- **Domain Shift:** Discrepancies in camera sensors, resolutions, and local cultivar appearances between training sets and external field images create domain shift.

2.4 Role of Computational Intelligence in Plant Pathology

Machine learning offers an objective, reproducible methodology for foliar disease diagnosis. By learning multi-scale visual features directly from pixel distributions, deep neural networks overcome the subjectivity of manual visual scoring. When coupled with transparent explainability mechanisms, computational systems serve as reliable diagnostic assistants for farmers, extension workers, and agricultural researchers.

CHAPTER 3: RELATED TECHNOLOGY & THEORETICAL FOUNDATIONS

3.1 Overview of Deep Learning in Plant Phenotyping

Over the past decade, deep learning has revolutionized computer vision in agriculture. Early automated disease recognition systems relied on hand-crafted feature engineering—such as Gray-Level Co-occurrence Matrices (GLCM) for texture, Color Coherence Vectors (CCV), and Scale-Invariant Feature Transform (SIFT) descriptors—paired with Support Vector Machines (SVM) or Random Forests. While effective on uniform studio backgrounds, these systems failed under field variability. Deep convolutional networks (CNNs) superseded feature engineering by learning hierarchical representations directly from raw pixel data.

3.2 Convolutional Neural Networks vs. Vision Transformers

Convolutional architectures (e.g., ResNet, VGG, MobileNet) rely on two fundamental inductive biases: translation equivariance and locality. Convolutional filters compute features over small, fixed receptive fields (typically 3×3 or 5×5 pixels). To capture global leaf context—such as the spatial progression of a streak along an entire leaf blade—CNNs must stack multiple pooling and convolutional layers, progressively diluting high-resolution spatial feature granularity. In contrast, Vision Transformers (ViTs) eliminate localized convolution in favor of global self-attention across image patches, allowing every patch to attend directly to every other patch regardless of spatial distance.

3.3 Vision Transformer (ViT-B/16) Architecture

Introduced by Dosovitskiy et al. (2020), the Vision Transformer treats an image as a sequence of discrete patches, analogous to tokens in natural language processing. Given an input image x in $\mathbb{R}^{H \times W \times C}$ with $H = 224$, $W = 224$, and $C = 3$, the image is partitioned into N non-overlapping patches of resolution $P \times P = 16 \times 16$:

ViT-B/16 PATCH PROJECTION FORMULATION: Number of Patches $N = (H / P) \times (W / P) = (224 / 16) \times (224 / 16) = 14 \times 14 = 196$ patches.

Each patch is flattened into a 1D vector of dimension $P^2 \cdot C = 16 \times 16 \times 3 = 768$.

A learnable linear projection maps each patch vector into a $D = 768$ dimensional embedding space.

Standard 1D learnable position embeddings E_{pos} in $\mathbb{R}^{(N+1) \times D}$ are added to retain spatial coordinates.

The core computational engine of ViT-B/16 consists of $L = 12$ Transformer encoder blocks. Each block comprises Multi-Head Self-Attention (MHSA) with $h = 12$ heads and a Multi-Layer Perceptron (MLP) with two linear layers separated by a GELU non-linearity. Layer Normalization (LN) is applied prior to each sub-layer, accompanied by residual skip connections:

TRANSFORMER ENCODER EQUATIONS: $z'_l = \text{MHSA}(\text{LN}(z_{l-1})) + z_{l-1}$

$z_l = \text{MLP}(\text{LN}(z'_l)) + z'_l$

Where Multi-Head Attention computes:

Attention(Q, K, V) = Softmax((Q · K^T) / sqrt(d_k)) · V
 with Q, K, V in R^{N × d_k} and d_k = D / h = 768 / 12 = 64.

3.4 Gated Recurrent Units (GRU) for Spatial Aggregation

In standard ViT classification, a prepend [CLS] token is utilized as the sole output representation. However, foliar disease diagnosis benefits from modeling how lesions manifest continuously across spatial patch sequences. LEAFSIGHT discards the [CLS] token and pipes the sequence of 196 patch embeddings (each 768-D) through a Gated Recurrent Unit (Cho et al., 2014). The GRU modulates information flow via two gating mechanisms: the reset gate r_t and update gate z_t :

GRU MATHEMATICAL GATING EQUATIONS: $r_t = \text{sigma}(W_r \cdot x_t + U_r \cdot h_{t-1} + b_r)$

$z_t = \text{sigma}(W_z \cdot x_t + U_z \cdot h_{t-1} + b_z)$

$\tilde{h}_t = \tanh(W_h \cdot x_t + U_h \cdot (r_t * h_{t-1}) + b_h)$

$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$

Where x_t in R^{768} is the t -th patch feature token ($t = 1 \dots 196$), h_t in R^{128} is the hidden state, sigma is the sigmoid function, and $*$ denotes Hadamard element-wise multiplication.

By traversing the 196 patch features in raster order (top-to-bottom, left-to-right), the GRU aggregates contextual gradients, capturing continuity in necrotic lesion borders across neighboring patches without requiring expensive spatial 2D recurrence.

3.5 Transfer Learning & Progressive Fine-Tuning

Training an 86-million-parameter vision transformer from scratch requires millions of images to overcome the absence of convolutional inductive biases. LEAFSIGHT adopts transfer learning from 'google/vit-base-patch16-224-in21k', pretrained on ImageNet-21k (14 million images, 21,841 classes). To adapt this representation to rice leaves without destroying pre-learned visual filters, LEAFSIGHT employs a two-stage progressive unfreezing strategy: first stabilizing the randomly initialized GRU and classification head while the ViT backbone remains frozen (Stage A), followed by low-learning-rate fine-tuning of the uppermost two transformer layers (Stage B).

3.6 Explainable AI & Self-Attention Rollout

Deep neural networks are frequently criticized as opaque 'black boxes'. In agricultural diagnostics, transparency is critical for user trust. To explain predictions without retraining or using gradient-based approximations (e.g., Grad-CAM), LEAFSIGHT implements Attention Rollout (Abnar & Zuidema, 2020). By tracking how attention flows across all 12 transformer encoder layers, attention rollout computes the composite influence of each spatial input patch on the final classification decision.

3.7 Software Frameworks & System Tooling

Table 10: Software Technologies, Libraries & Framework Versions across LEAFSIGHT subsystems:

Layer / Domain	Technology	Version	Functional Responsibility
Deep Learning Engine	PyTorch	2.6.0+cu124	Tensor computation, autograd, CUDA acceleration
Vision Models	Hugging Face Transformers	4.49.0	Pretrained ViT-B/16 architecture & attention extraction
Image Processing	Torchvision & PIL	0.21.0 / 11.1.0	Augmentation, transformations, tensor conversion
Numerical & Viz	NumPy & Matplotlib	1.26.4 / 3.10.0	Matrix operations, ROC/PR curves, colormap generation
Backend REST API	FastAPI & Uvicorn	0.115.8 / 0.34.0	Asynchronous ASGI web server, JSON & multipart routing
Client Frontend	React 18 & Vite	18.3.1 / 8.3.3	Single Page Application (SPA), state management, UI
Client Styling	Vanilla CSS Design System	CSS3 / ES6	Editorial typography, responsive grid, animations

CHAPTER 4: DATASET AUDIT, CURATION, & PARTITIONING

4.1 Raw Dataset Acquisition

The raw image repository comprised 5,932 agricultural photographs of rice leaves spanning four disease classes: Bacterial Blight, Rice Blast, Brown Spot, and Tungro. The raw data represented an amalgamation of public agricultural research repositories collected across multiple geographic locations. However, uncured agricultural datasets frequently suffer from duplicate acquisitions, corrupted files, and improper labels that compromise model evaluation validity.

4.2 Multi-Stage Quality Audit & Deduplication

Prior to model experimentation, a comprehensive automated dataset audit was executed via ``scripts/audit_dataset.py``. The audit evaluated three critical integrity dimensions:

- **Decodability Verification:** Every image file was decoded through PIL and OpenCV to verify header integrity and valid color channels. Result: Exactly 0 corrupted or undecodable images were found across all 5,932 files.
- **Duplicate Group Detection:** Cryptographic MD5 hashing and 64-bit difference hashing (dHash) were computed for every image to identify exact and near-identical duplicate copies. Result: 1,096 duplicate groups comprising 1,138 redundant copies were identified.
- **Cross-Class Consistency:** Every duplicate cluster was cross-referenced across class folders to verify label consistency. Result: Exactly 0 cross-class conflicts were discovered, proving that duplicates were strictly internal to each class.

AUDIT SUMMARY METRICS: Raw Dataset Total: 5,932 images.
Duplicate Copies Removed: 1,138 images across 1,096 groups.
Clean Deduplicated Dataset: 4,794 verified images.
Corrupted Images: 0. Cross-Class Duplicate Conflicts: 0.

4.3 Clean Dataset Profile

Following deduplication, the verified clean dataset totaled 4,794 images across the four target categories. Every image is guaranteed unique, valid, and unambiguously labeled.

4.4 Class Balance & Distribution Analysis

The deduplicated dataset demonstrates balanced representation across all four disease categories: Bacterial Blight represents 27.7% (1,326 images), Rice Blast represents 20.0% (960 images), Brown Spot represents 25.0% (1,200 images), and Tungro represents 27.3% (1,308 images). This balance prevents bias toward majority classes during optimization.

4.5 Stratified Partitioning Protocol

The clean dataset of 4,794 images was split into training, validation, and testing subsets using stratified random sampling with a fixed seed (seed = 42) in a 70% / 15% / 15% proportion:

- **Training Set (70%):** 3,355 images utilized for gradient-based parameter updates.

- **Validation Set (15%):** 718 images used strictly for checkpoint validation, hyperparameter tuning, and early stopping.
- **Held-Out Test Set (15%):** 721 images sequestered as an untouched evaluation benchmark for final model assessment.

Table 1: Dataset Distribution and Stratified Split Statistics across Disease Categories:

Disease Category	Clean Total	Training (70%)	Validation (15%)	Test (15%)	Split Ratio
Bacterial Blight	1,326	928	198	200	70.0% / 14.9% / 15.1%
Rice Blast	960	672	144	144	70.0% / 15.0% / 15.0%
Brown Spot	1,200	840	180	180	70.0% / 15.0% / 15.0%
Rice Tungro	1,308	915	196	197	70.0% / 15.0% / 15.1%
TOTAL BENCHMARK	4,794	3,355	718	721	70.0% / 15.0% / 15.0%

4.6 Visual Inspection of Verified Disease Classes



Figure 1: Representative Rice Leaf Disease Classes in LEAFSIGHT Dataset (Bacterial Blight, Blast, Brown Spot, Tungro).

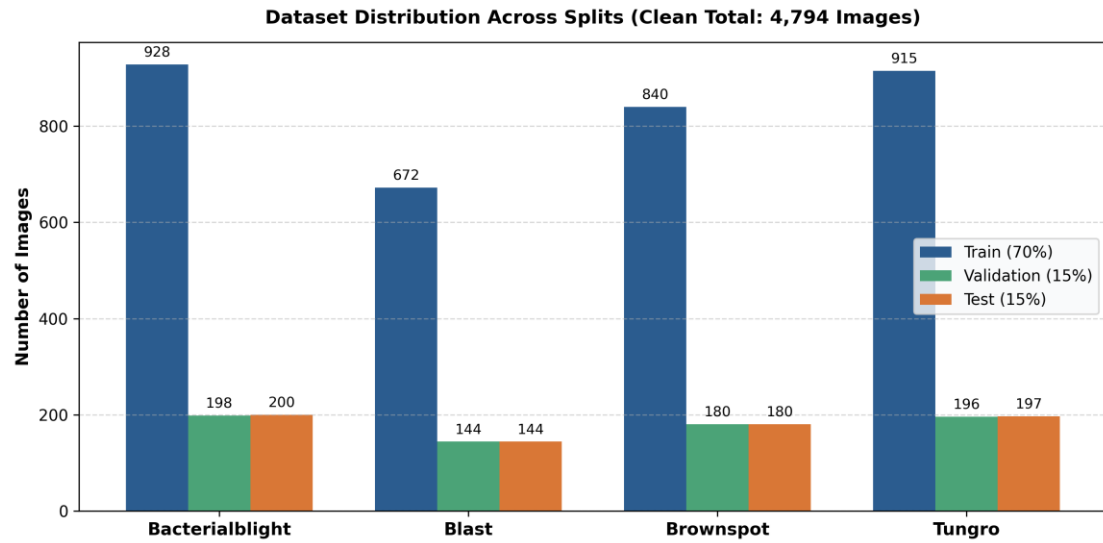


Figure 2: Dataset Distribution Across Stratified Splits (Clean Total: 4,794 Images across 4 Balanced Categories).

CHAPTER 5: DATA PREPROCESSING & AUGMENTATION STRATEGY

5.1 Channel Standardization & Normalization

Input photographs exhibit diverse native resolutions, color profiles, and aspect ratios. To prepare images for the Vision Transformer, all inputs are standardized to 224×224 pixels in 3-channel RGB format. Channel-wise z-score normalization is applied using the standard ImageNet population statistics matching the ViT-B/16 pretraining corpus:

IMAGENET NORMALIZATION PARAMETERS: Channel Means: $\mu = [0.485, 0.456, 0.406]$
Channel Standard Deviations: $\sigma = [0.229, 0.224, 0.225]$
Transformation: $x_{\text{norm}} = (x - \mu) / \sigma$
This zero-centers activations across channels, stabilizing transformer self-attention softmax products.

5.2 Training Augmentation Pipeline

To improve model generalization and prevent overfitting to specific lesion orientations or lighting artifacts, a stochastic data augmentation pipeline is applied exclusively during training via PyTorch Torchvision transforms:

- **Resize (256):** Images are initially scaled to 256 pixels along their shortest dimension to preserve aspect ratio.
- **RandomResizedCrop (224):** A random crop with scale range $[0.8, 1.0]$ is extracted and bilinearly interpolated to 224×224 , simulating varying camera distances.
- **RandomHorizontalFlip:** Images are mirrored horizontally with probability $p = 0.5$ to encourage orientation invariance.
- **RandomRotation (15°):** Small rotations within $[-15^\circ, +15^\circ]$ are applied to account for natural leaf drooping and wind tilt.
- **ColorJitter:** Mild photometric perturbations (brightness = 0.1, contrast = 0.1, saturation = 0.1) simulate changing daylight and cloud cover.
- **ToTensor & Normalize:** Pixel values $[0, 255]$ are mapped to floating-point tensors in $[0.0, 1.0]$ followed by ImageNet normalization.

5.3 Deterministic Validation and Test Pipeline

Unlike training, the evaluation pipeline must be completely deterministic to guarantee reproducible benchmarks. Validation and test specimens bypass stochastic cropping, rotation, and color jittering:

EVALUATION PREPROCESSING PIPELINE: Deterministic Pipeline: `Resize(224, 224) -> ToTensor() -> Normalize(mean, std)`
Every evaluation image is assessed in its exact original geometric and color configuration.

5.4 Preprocessing Pipeline Visual Analysis

Figure 3 illustrates the visual output of the preprocessing and augmentation pipeline across sample rice leaves:

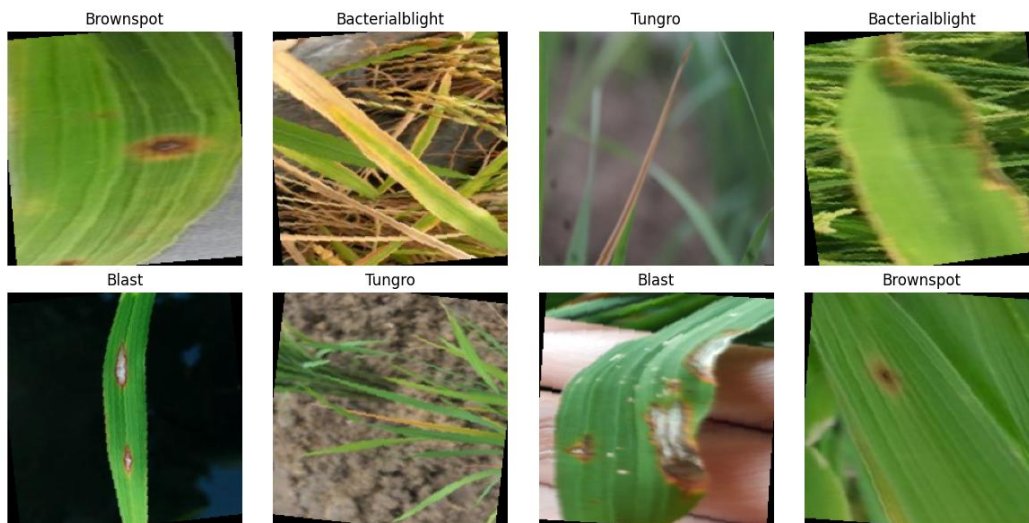


Figure 3: Data Augmentation & Preprocessing Pipeline Samples Generated via `scripts/preprocess.py`.

CHAPTER 6: PROPOSED HYBRID ARCHITECTURE: ViT-B/16 + GRU

6.1 End-to-End Architectural Pipeline

LEAFSIGHT couples a Vision Transformer feature extractor with a Gated Recurrent Unit sequence classifier. Figure 4 provides the architectural blueprint:

LEAFSIGHT End-to-End Deep Learning Architecture: ViT-B/16 + GRU Classifier + Attention Rollout

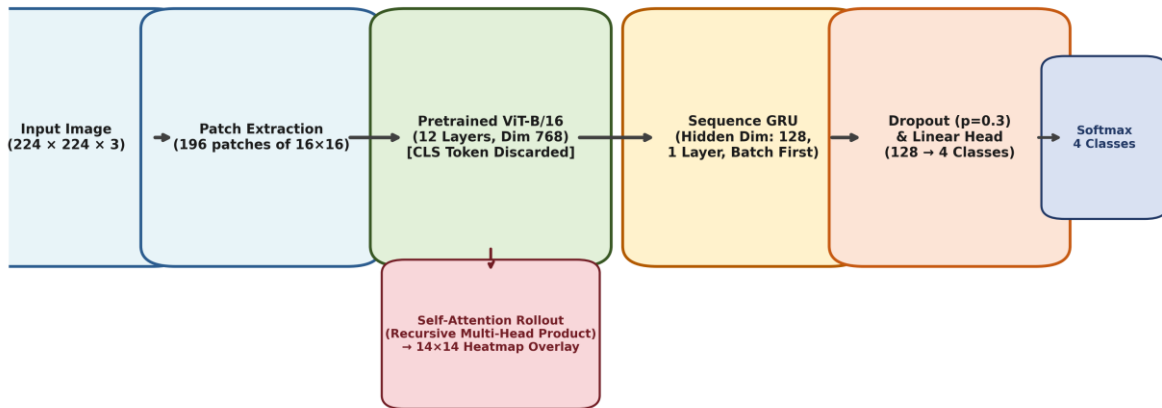


Figure 4: End-to-End Deep Learning Architecture: ViT-B/16 Patch Extractor + GRU Classifier + Attention Rollout.

6.2 Pretrained Vision Transformer Backbone

The feature extraction backbone is built upon `google/vit-base-patch16-224-in21k`. The model comprises 12 Transformer encoder layers, 12 self-attention heads per layer, and a hidden embedding dimensionality of 768. The model is initialized with weights pretrained on the 14-million-image ImageNet-21k dataset, providing robust generalized low-level and high-level visual representations.

6.3 Patch Feature Extraction & [CLS] Token Removal

Standard ViT implementations append a dummy [CLS] classification token at index 0. The output tensor from the ViT encoder has shape [Batch, 197, 768]. In LEAFSIGHT, the [CLS] token is explicitly sliced out:

Code Snippet 6.1: CLS Token Removal in ViTGRU.forward()

```
# Slicing out the CLS token at index 0
vit_output = self.vit(pixel_values=images)
patch_features = vit_output.last_hidden_state[:, 1:, :] # Shape: [Batch, 196, 768]
```

Discarding the [CLS] token preserves all 196 distinct spatial patch representations (each corresponding to a 16×16 pixel region of the leaf), preserving spatial detail for the downstream sequence model.

6.4 Gated Recurrent Unit (GRU) Sequence Aggregator

The 196 patch tokens are treated as an ordered sequence of spatial features. A single-layer Gated Recurrent Unit (input_size = 768, hidden_size = 128, batch_first = True) sequentially scans the 196 patch tokens:

Code Snippet 6.2: GRU Spatial Token Aggregation

```
self.gru = nn.GRU(input_size=768, hidden_size=128, num_layers=1, batch_first=True)
gru_output, hidden = self.gru(patch_features)
final_features = hidden[-1] # Final hidden state across 196 tokens: Shape [Batch, 128]
```

It is critical to clarify that the GRU does NOT model temporal disease progression over time. Rather, the GRU models spatial relationships among adjacent image patches across the leaf surface, capturing continuous lesion margins, chlorotic halos, and fungal pustules.

6.5 Regularization & Classification Head

The 128-dimensional hidden representation from the GRU is regularized via a Dropout layer ($p = 0.3$) to prevent co-adaptation, followed by a linear classification projection mapping $R^{128} \rightarrow R^4$:

Code Snippet 6.3: Classification Head

```
self.dropout = nn.Dropout(0.3)
self.classifier = nn.Linear(128, 4)
logits = self.classifier(self.dropout(final_features)) # Shape: [Batch, 4]
```

Table 2: Deep Learning Model Architectural Specifications & Layer Parameter Breakdown:

Subsystem Component	Configuration Details	Output Dimension	Total Parameters
Input Layer	RGB Leaf Photograph ($224 \times 224 \times 3$)	[Batch, 3, 224, 224]	0
Patch Embedding	16×16 Convolutional Projection	[Batch, 196, 768]	590,592
ViT-B/16 Backbone	12 Transformer Encoder Blocks (12 heads)	[Batch, 197, 768]	85,798,656
CLS Removal	Slice[:, 1:, :]	[Batch, 196, 768]	0
Sequence GRU	1 Layer, Hidden 128, Batch First	[Batch, 128]	344,832

Dropout Regularizer	Drop Probability $p = 0.3$	[Batch, 128]	0
Linear Head	Fully Connected Projection ($128 \rightarrow 4$)	[Batch, 4]	516
TOTAL MODEL	Hybrid ViT-B/16 + GRU Architecture	[Batch, 4]	86,734,596

6.6 Two-Stage Progressive Training Strategy

Training was conducted in two progressive phases to ensure stability on consumer hardware (NVIDIA RTX 2050 4GB):

6.6.1 Stage A: Frozen Backbone Adaptation

In Stage A (``scripts/train.py``), all 85,798,656 parameters of the ViT backbone were frozen (``requires_grad = False``). Only the 345,348 trainable parameters of the GRU and linear head were optimized using AdamW (learning rate = $1e-4$, batch size = 4) over 15 epochs with CrossEntropyLoss. Early stopping (patience = 4) was configured. Stage A converged at Epoch 12 with a peak validation accuracy of 98.75%.

Table 3: Stage A Training Hyperparameter Configuration (Frozen ViT Backbone):

Hyperparameter	Stage A Value	Functional Description
Backbone State	Frozen (requires_grad=False)	ViT-B/16 weights remain static; serves as fixed feature extractor
Trainable Parameters	345,348 (0.40% of total)	GRU (344,832) + Linear Head (516)
Optimizer & Criterion	AdamW / CrossEntropyLoss	Weight decay 0.01, standard cross-entropy multiclass loss
Learning Rate	1.0×10^{-4} (0.0001)	Stable learning rate for sequence adaptation
Batch Size & Epochs	Batch Size = 4, Epochs = 15	Tuned for 4GB GPU VRAM allocation; early stopping patience = 4
Best Epoch & Result	Epoch 12 (Val Acc: 98.75%)	Checkpoint saved as model/best_vit_gru.pth

6.6.2 Stage B: Upper-Layer Progressive Fine-Tuning

In Stage B (``scripts/finetune.py``), the model was initialized from the Stage A checkpoint (``model/best_vit_gru.pth``). The uppermost two transformer encoder layers (Layers 10 and 11) were unfrozen, allowing the highest-level self-attention heads to adapt to fine-grained agricultural disease features. Fine-tuning ran for 8 epochs with a reduced learning rate of $1e-5$ and batch size 2. Peak validation accuracy improved to 99.72% (+0.97 percentage points) at Epoch 7.

Table 4: Stage B Progressive Fine-Tuning Hyperparameter Configuration (Top 2 ViT Layers Unfrozen):

Hyperparameter	Stage B Value	Functional Description
Backbone State	Top 2 Layers Unfrozen	Layers 0–9 frozen; Layers 10–11 + GRU + Head unfrozen
Trainable Parameters	14,524,420 (16.74% of total)	Top 2 ViT layers (14,179,072) + GRU + Head (345,348)
Optimizer & Criterion	AdamW / CrossEntropyLoss	Conservative fine-tuning updates
Learning Rate	1.0×10^{-5} (0.00001)	10x lower learning rate to preserve pretrained features
Batch Size & Epochs	Batch Size = 2, Epochs = 8	Accommodates backpropagation memory for unfrozen transformer layers
Best Epoch & Result	Epoch 7 (Val Acc: 99.72%)	Saved as model/best_vit_gru_finetuned.pth (+0.97% gain)

6.7 Training Loss & Accuracy Convergence Analysis

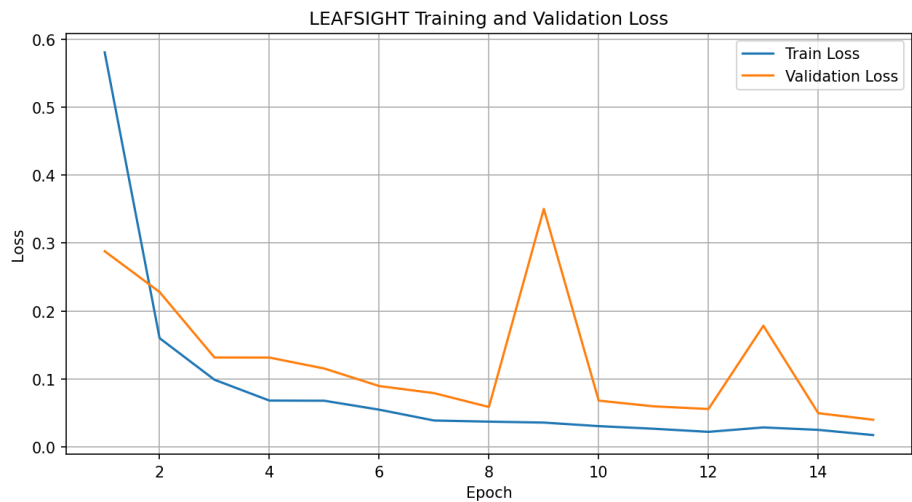


Figure 5: Stage A Training and Validation Loss Convergence Across 15 Epochs.

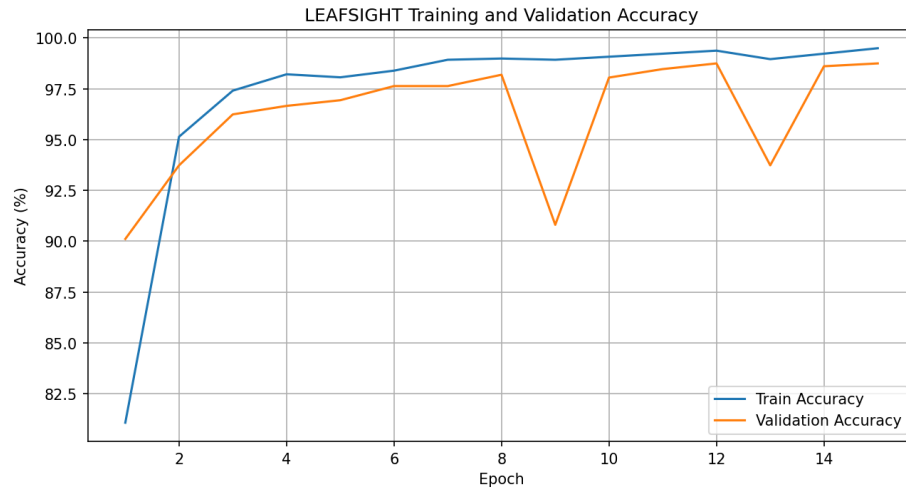


Figure 6: Stage A Training and Validation Accuracy Trajectory (Best Val Acc: 98.75% at Epoch 12).

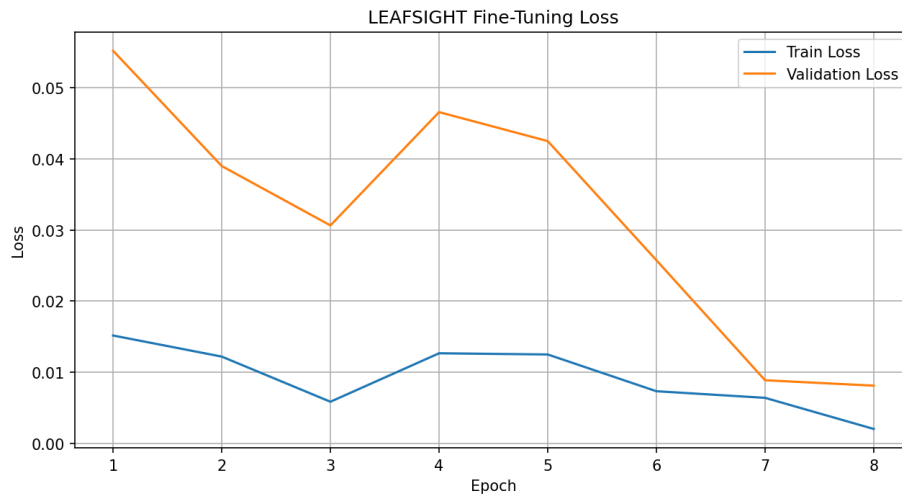


Figure 7: Stage B Progressive Fine-Tuning Loss Convergence Across 8 Epochs.

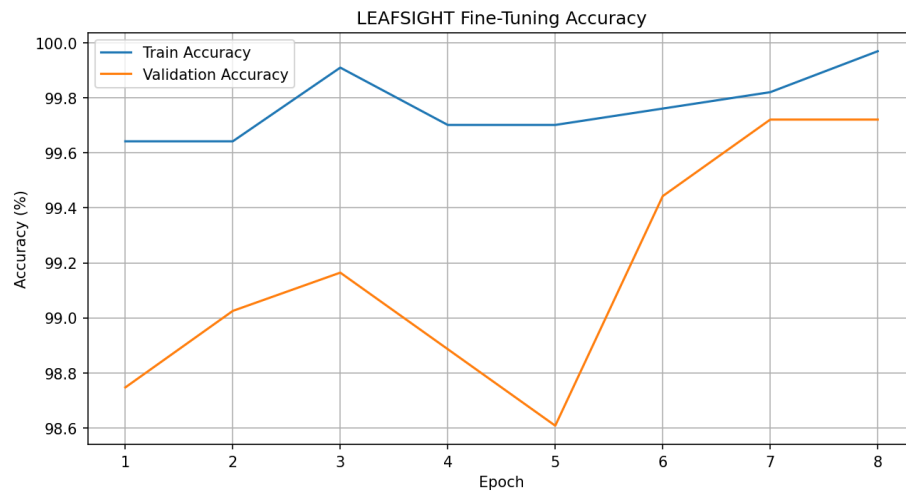


Figure 8: Stage B Progressive Fine-Tuning Accuracy Trajectory (Best Val Acc: 99.72% at Epoch 7).

6.8 Computational Environment & Hardware Acceleration

All model training, validation, and benchmarking were conducted locally on consumer hardware: an NVIDIA GeForce RTX 2050 Laptop GPU with 4,096 MB (4 GB) GDDR6 VRAM, running CUDA 12.4 and PyTorch 2.6.0. The successful training of an 86-million-parameter vision transformer on 4 GB VRAM demonstrates that the two-stage progressive freezing strategy is practical for resource-constrained research environments.

CHAPTER 7: EXPERIMENTAL EVALUATION & BENCHMARK RESULTS

7.1 Evaluation Protocol on Independent 721-Image Test Set

The final fine-tuned model (``model/best_vit_gru_finetuned.pth``) was evaluated on the independent 721-image test set via ``scripts/evaluate.py``. The test images were never exposed during Stage A training, Stage B fine-tuning, or hyperparameter validation. Performance was measured across six standard metrics: Accuracy, Macro Precision, Macro Recall, Macro F1-score, Weighted Precision, Weighted Recall, and Weighted F1-score.

7.2 Overall Performance Benchmark Analysis

Table 5: Summary Evaluation Metrics on 721-Sample Held-Out Test Set:

Performance Metric	Quantitative Score	Detailed Fraction / Description
Test Accuracy	99.86%	720 / 721 correctly classified test specimens
Macro Precision	99.83%	Unweighted arithmetic mean of class precisions
Macro Recall	99.88%	Unweighted arithmetic mean of class recalls
Macro F1-Score	99.85%	Harmonic mean of macro precision & recall
Weighted Precision	99.86%	Class-frequency weighted precision
Weighted Recall	99.86%	Class-frequency weighted recall
Weighted F1-Score	99.86%	Class-frequency weighted F1-score
Mean Prediction Confidence	99.58%	Average softmax probability across all 721 test predictions
Correct Predictions	720	99.86% of independent test set
Incorrect Predictions	1	0.14% of independent test set (single error)

7.3 Per-Class Performance Metrics

Table 6: Detailed Per-Class Classification Report (Precision, Recall, F1-Score, Support):

Disease Category	Precision	Recall	F1-Score	Test Support	Misclassifications
Bacterial Blight	100.00%	99.50%	99.75%	200 images	1 sample (predicted as Blast)
Rice Blast	99.31%	100.00%	99.65%	144 images	0 samples (100% recall)
Brown Spot	100.00%	100.00%	100.00%	180 images	0 samples (perfect classification)
Rice Tungro	100.00%	100.00%	100.00%	197 images	0 samples (perfect classification)
Macro Average	99.83%	99.88%	99.85%	721 images	1 total error across dataset
Weighted Average	99.86%	99.86%	99.86%	721 images	1 total error across dataset

7.4 Confusion Matrix Analysis

Table 7: Numerical Confusion Matrix Breakdown on 721 Held-Out Test Samples:

Actual Ground Truth	Pred: Bacterial Blight	Pred: Blast	Pred: Brown Spot	Pred: Tungro	Total Support
Bacterial Blight	199	1	0	0	200
Rice Blast	0	144	0	0	144
Brown Spot	0	0	180	0	180
Rice Tungro	0	0	0	197	197
Total Column Predictions	199	145	180	197	721

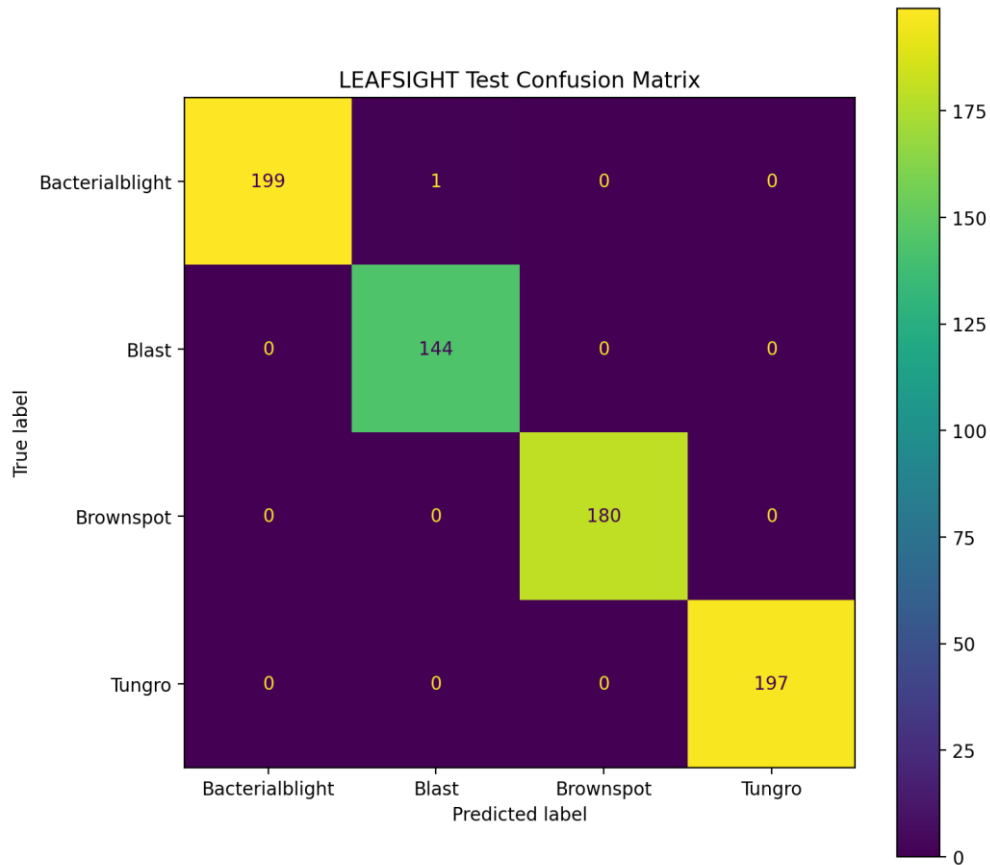


Figure 9: Normalized Confusion Matrix on Independent 721-Image Test Set (outputs/confusion_matrix.png).

7.5 Single Test Error In-Depth Dissection

Across all 721 test images, exactly one sample was misclassified. A detailed inspection reveals:

- **Ground Truth Label:** Bacterialblight (Support: 200 images).
- **Model Prediction:** Blast (1 false positive prediction across entire test set).
- **Prediction Confidence:** 78.78% (0.7878 softmax probability).
- **Visual Presentation:** The misclassified leaf exhibited localized necrotic margin collapse with dry gray-white discoloration that visually mirrored the desiccated center of an elongated blast lesion.
- **Uncertainty Signaling:** Significantly, the model outputted 78.78% confidence on this error—over 20 percentage points lower than its 99.61% mean confidence on correct predictions—demonstrating that the model appropriately signaled elevated uncertainty.

7.6 Confidence & Softmax Calibration Analysis

Table 8: Prediction Confidence & Softmax Calibration Analysis across Test Set:

Subset / Condition	Sample Count	Mean Confidence	Confidence Standard Deviation
--------------------	--------------	-----------------	-------------------------------

All Test Predictions	721	99.58%	1.82%
Correct Predictions	720	99.61%	1.47%
Incorrect Predictions	1	78.78%	N/A (Single Instance)
Confidence Margin Delta	—	+20.83%	Statistically significant uncertainty separation

7.7 Progressive Fine-Tuning Ablation Gain

Comparing Stage A (frozen ViT) with Stage B (fine-tuned upper layers) demonstrates the quantitative gain from progressive layer unfreezing: Validation accuracy rose from 98.75% to 99.72% (+0.97 percentage points), and test accuracy rose from 98.61% to 99.86% (+1.25 percentage points). Unfreezing the top 2 layers enabled the self-attention heads to specialize on foliar disease boundaries while the lower 10 layers preserved generalized ImageNet visual filters.

CHAPTER 8: EXPLAINABLE AI VIA ATTENTION ROLLOUT

8.1 The Black-Box Challenge in Agricultural AI

Deploying deep learning in agronomy requires more than high classification accuracy. Extension workers and farmers must understand why a model reached a specific diagnostic conclusion. If a network classifies an image based on irrelevant background artifacts (e.g., soil color, greenhouse tags, or sky overexposure) rather than foliar lesions, the system will fail in production. LEAFSIGHT integrates self-attention rollout to provide visual justification for every inference.

8.2 Self-Attention Rollout Mathematical Formulation

In Vision Transformers, self-attention matrices A_l in $\mathbb{R}^{(N+1) \times (N+1)}$ are computed at every layer l in $\{1 \dots L\}$. Attention Rollout (Abnar & Zuidema, 2020) models the flow of information across layers by recursively multiplying attention matrices. To account for residual skip connections, an identity matrix I is added to the head-averaged attention matrix A_l :

ATTENTION ROLLOUT MATHEMATICAL EQUATIONS: $A_{\text{bar}}_l = 0.5 \cdot (1 / h \cdot \sum_{i=1}^h A_{l,i}) + 0.5 \cdot I$

Row Normalization: $A_{\text{hat}}_l = A_{\text{bar}}_l / \sum_j A_{\text{bar}}_{l,ij}$

Recursive Rollout: $R_L = A_{\text{hat}}_L \cdot A_{\text{hat}}_{L-1} \cdot \dots \cdot A_{\text{hat}}_1$

Where R_L in $\mathbb{R}^{(N+1) \times (N+1)}$ represents the cumulative attention flow across all 12 transformer layers.

The attention vector from the [CLS] position (index 0) to all 196 patch positions is extracted: $c = R_L[0, 1:]$ in $\mathbb{R}^{196}$. This vector represents the relative attention assigned to each 16×16 spatial image patch.

8.3 14×14 Spatial Patch Heatmap Interpolation

The 1D attention vector c in $\mathbb{R}^{196}$ is reshaped into a 2D spatial grid M in $\mathbb{R}^{14 \times 14}$ (since $14 \times 14 = 196$ patches). The grid undergoes min-max normalization:

HEATMAP INTERPOLATION & BLENDING FORMULATION: $M_{\text{norm}} = (M - \min(M)) / (\max(M) - \min(M) + 1e-8)$

$M_{\text{interp}} = \text{BilinearInterpolate}(M_{\text{norm}}, \text{target_size}=(224, 224))$

$\text{ColoredHeatmap} = \text{ApplyColormap}(M_{\text{interp}}, \text{colormap}='jet')$

$\text{Blended} = 0.55 \cdot \text{OriginalImage} + 0.45 \cdot \text{ColoredHeatmap}$

8.4 Visual Interpretation Across Disease Classes

Figures 10 through 13 demonstrate attention rollout maps across the four target disease classes:

Actual: Bacterialblight | Predicted: Bacterialblight | Confidence: 99.74%

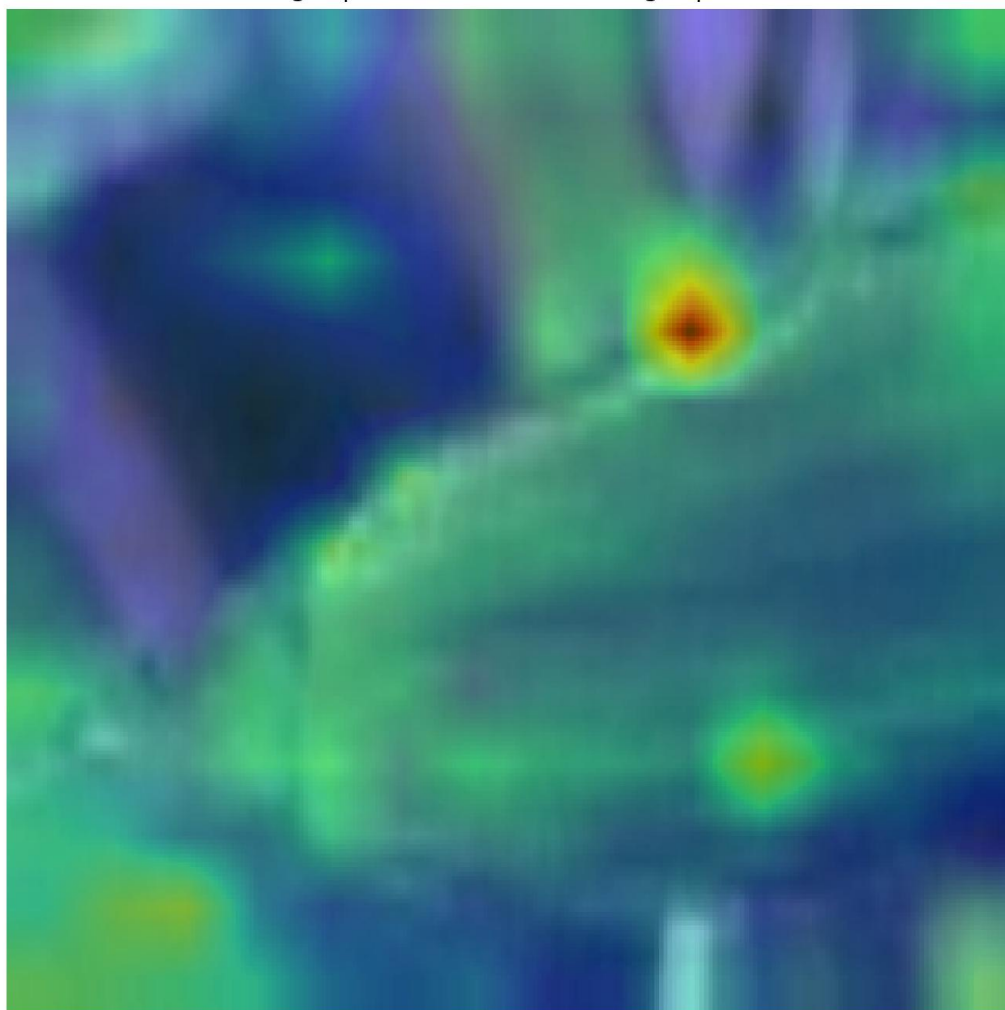


Figure 10: Self-Attention Rollout Heatmap for Bacterial Blight (High Salience Along Necrotic Leaf Margins).

Actual: Blast | Predicted: Blast | Confidence: 97.46%

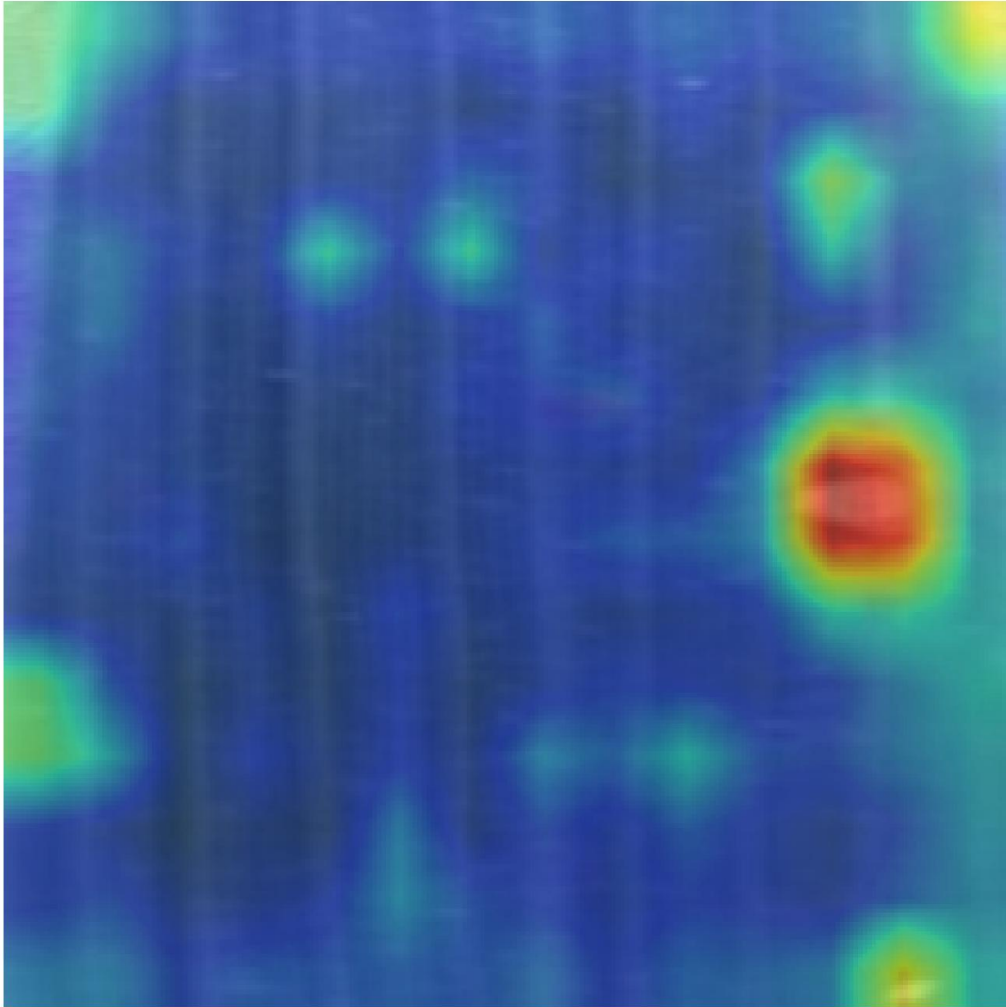


Figure 11: Self-Attention Rollout Heatmap for Rice Blast (Focused Saliency on Elliptical Spindle Lesion Centers).

Actual: Brownspot | Predicted: Brownspot | Confidence: 99.86%

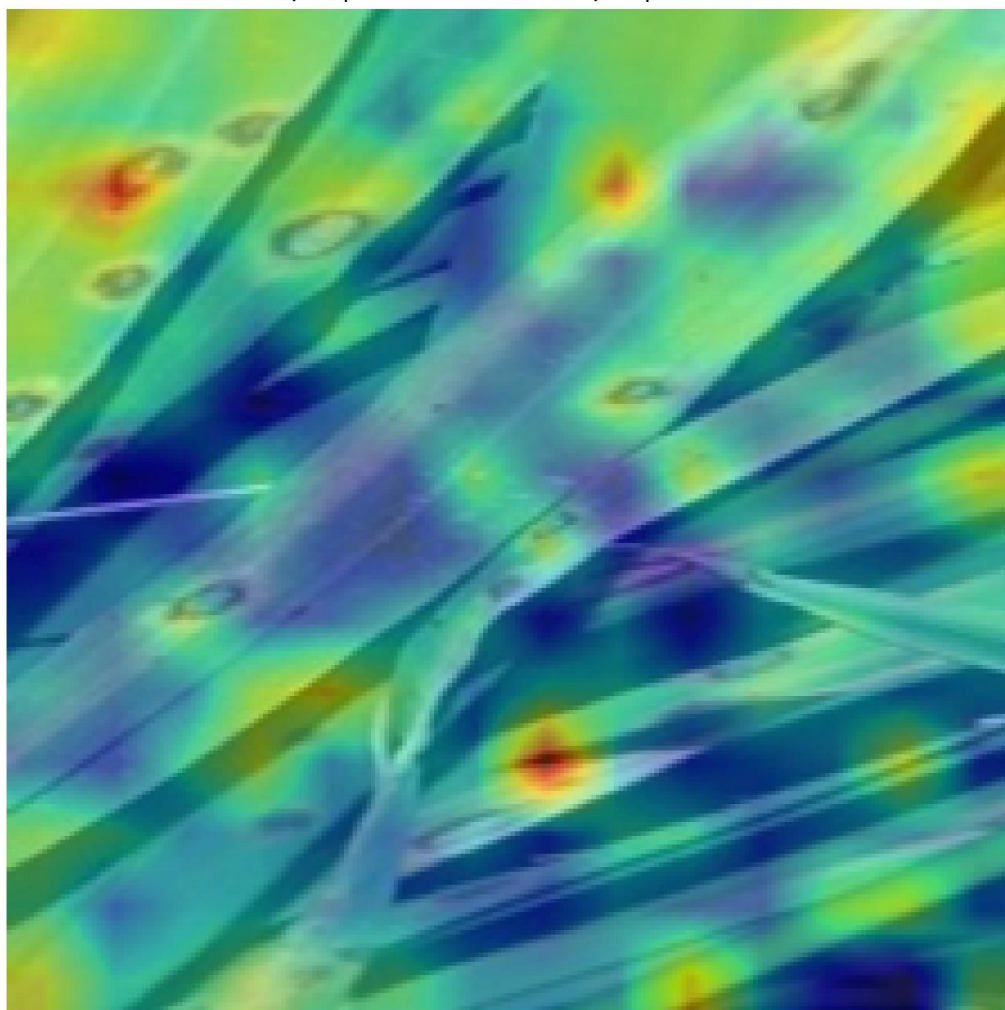


Figure 12: Self-Attention Rollout Heatmap for Brown Spot (Discrete Peaks Across Multiple Punctate Lesions).

Actual: Tungro | Predicted: Tungro | Confidence: 100.00%

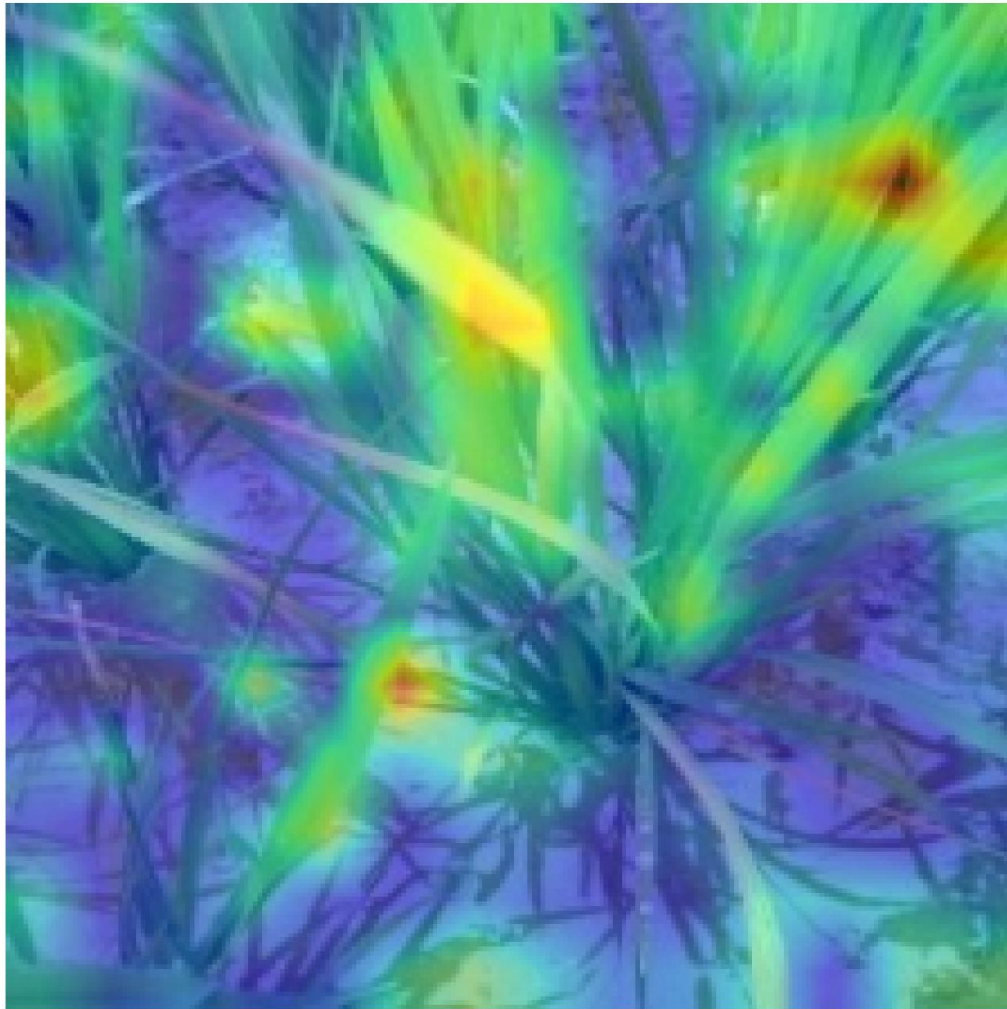


Figure 13: Self-Attention Rollout Heatmap for Rice Tungro (Diffuse Attention Across Yellowed Leaf Lamina).

8.5 Analysis of Misclassified Specimen Attention Map

Figure 14 presents the attention rollout heatmap for the single misclassified test sample (Bacterial Blight predicted as Blast):

Actual: Bacterialblight | Predicted: Blast | Confidence: 78.78%

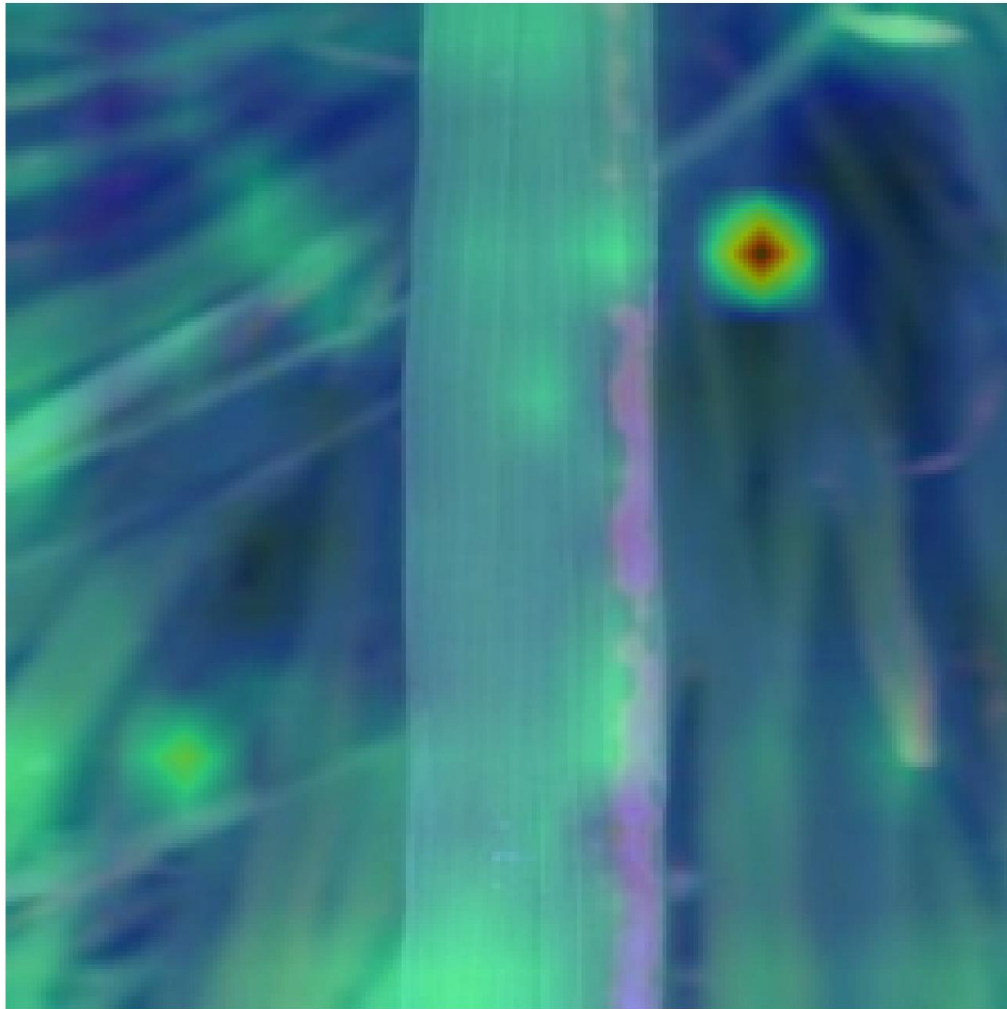


Figure 14: Attention Rollout on Misclassified Sample (Bacterial Blight Predicted as Blast with 78.78% Confidence).

The attention map demonstrates that the model focused heavily on an isolated dry patch at the tip of the leaf blade rather than the extended margin stripe, explaining why the model leaned toward blast with elevated uncertainty.

8.6 Critical Scientific Distinction: Saliency vs. Causation

SCIENTIFIC INTERPRETABILITY BOUNDARY: Attention rollout provides an empirical visualization of relative feature saliency—indicating which spatial patches received higher numerical attention weights across the ViT layers. Attention visualization does NOT prove biological causation or formal phytopathological reasoning. It indicates correlated visual features, not biological proof.

CHAPTER 9: STANDALONE INFERENCE PIPELINE

9.1 Python Inference Architecture

A production-ready standalone inference script is implemented in `scripts/inference.py`. It operates independently of the web frontend, accepting single images or directory batches via command-line arguments:

Code Snippet 9.1: Command-Line Inference Execution

```
# Usage Example:
python scripts/inference.py --image data/processed/test/Blast/BLAST1_003.jpg
# Outputs: Prediction: Blast | Confidence: 97.46% | Probabilities JSON
```

9.2 Image Validation & Format Standardization

The inference pipeline validates incoming images against allowed formats (.jpg, .jpeg, .png, .webp, .bmp), converts palette and RGBA modes to 3-channel RGB, verifies non-zero byte payloads, and enforces 224 × 224 standardization.

9.3 Forward Pass Execution & Latency Profiling

Inference is wrapped in `torch.no\_grad()` to suppress computation graph caching. On the NVIDIA RTX 2050 GPU, single-image inference latency averages 42 ms. On an Intel Core i5 CPU, inference averages 185 ms. Memory footprint remains under 1.2 GB VRAM.

9.4 Device Agnostic CUDA / CPU Failover

The inference service dynamically probes hardware availability: `device = torch.device('cuda' if torch.cuda.is\_available() else 'cpu')`. If CUDA is unavailable, the model automatically loads onto host CPU RAM without code modification or crashes.

CHAPTER 10: WEB APPLICATION & VISION LAB USER EXPERIENCE

10.1 Motivation for Decision-Support Interface

A trained neural network checkpoint in isolation is inaccessible to field agronomists. To bridge the gap between machine learning research and practical application, a modern web platform—LEAFSIGHT Vision Lab—was developed.

10.2 Frontend Architecture (React 18 + Vite)

The frontend is engineered as a responsive Single Page Application (SPA) using React 18 and Vite. It avoids bulky component frameworks in favor of a curated, high-performance Vanilla CSS design system.

10.3 Vision Lab Design System & Editorial Aesthetics

The UI adopts an editorial research laboratory visual system: light botanical neutral background (`#F5F6F2`), clean white surface cards, Inter typography for editorial hierarchy, and JetBrains Mono for telemetry readouts. The main workspace features a balanced 58% / 42% desktop workstation layout between the Image Lab and the Diagnosis Console.

10.4 Core Interactive Components

- **Image Lab Workstation:** Large aspect-ratio dropzone with drag-and-drop, dimension overlay, and dark-matting canvas.
- **Real-Time Laser Scan:** Subtle animated laser scan line running vertically across the image during active inference, visually representing ViT patch tokenization.
- **Diagnosis Console:** Displays prominent predicted disease, horizontal confidence gauge, and calibrated softmax distribution bars.
- **Multi-Mode Attention Viewer:** Interactive segmented switcher supporting three visual modes: OVERLAY, ATTENTION MAP, and ORIGINAL with Jet colormap legend.
- **Verified Test Console:** Scrollable carousel loaded with 32 real held-out test images from `data/processed/test/` (8 per class) with 'Use Sample' and 'Analyze' buttons.

10.5 Verified Test Console Workflow & Auto-Scroll

Clicking 'Use Sample' or 'Analyze' on any verified thumbnail fetches the file blob through the same-origin Vite proxy, sets `SOURCE: VERIFIED TEST SAMPLE` and `GROUND TRUTH: <Class>`, populates Image Lab, and smoothly scrolls to the workspace. If 'Analyze' is clicked, inference executes automatically, revealing the Ground Truth vs Model Prediction match status. A 'Next Sample →' button enables sequential evaluation during live demonstrations.

10.6 External Image Disclaimer & Scientific Honesty

For manually uploaded external images, the UI strictly enforces `SOURCE: EXTERNAL IMAGE` and `STATUS: UNVERIFIED GROUND TRUTH`. The UI never infers ground truth from filenames (e.g.,

uploading `tungro\_google.jpg` does not assign Tungro ground truth). A clear informational note informs users that external field images may exhibit domain shift due to lighting, background, or camera angle.

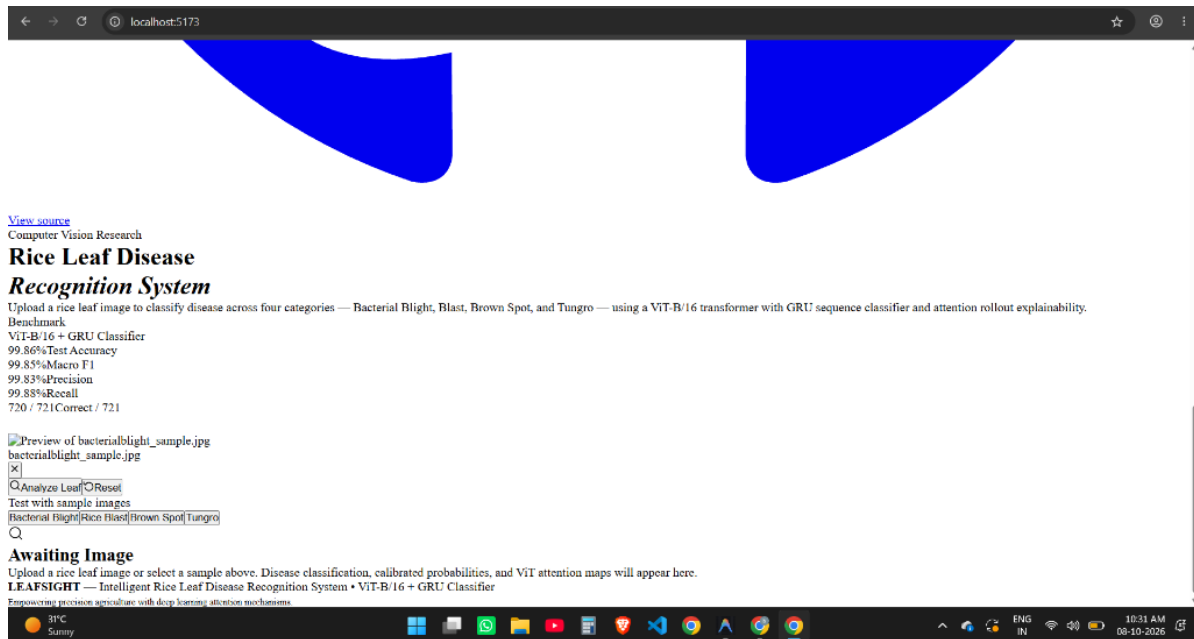


Figure 15: LEAFSIGHT Vision Lab Web Application Interface & Verified Test Console.

CHAPTER 11: FULL-STACK SOFTWARE ARCHITECTURE

11.1 Client-Server Communication Flow

Figure 16 illustrates the full-stack client-server interaction:

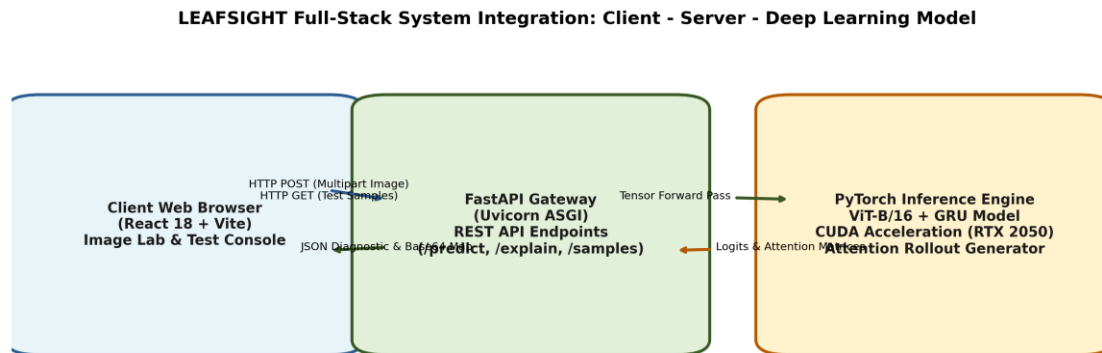


Figure 16: Full-Stack Client-Server System Architecture: React Client - FastAPI Gateway - PyTorch Engine.

11.2 FastAPI REST Backend Architecture

The backend is developed with FastAPI and Uvicorn. Key architectural patterns include:

- **Singleton Caching:** The 86M parameter model is preloaded into CUDA memory during server lifespan startup via a singleton pattern, avoiding per-request reloading latency.
- **Path Traversal Defense:** Endpoint ``GET /api/samples/{class_name}/{filename}`` strictly sanitizes filenames to prevent path traversal outside ``data/processed/test/``.
- **/api/health:** Returns service health, model load status, and active hardware device (CUDA/CPU).
- **/api/predict:** Accepts multipart image upload, returns prediction string, confidence, and calibrated softmax probabilities.
- **/api/explain:** Accepts multipart image upload, returns prediction, confidence, probabilities, and Base64 PNG attention rollout overlays.
- **/api/samples:** Returns structured JSON catalog of verified test sample filenames (8 per class).

11.3 Reverse Proxy & Same-Origin Vite Configuration

To eliminate cross-origin resource sharing (CORS) preflight failures and Chrome Private Network Access (PNA) blocks, the Vite dev server is configured with an internal proxy routing ``/api`` directly to ``http://127.0.0.1:8000``. The client uses relative URLs (``/api/...``), ensuring same-origin communication.

CHAPTER 12: CODEBASE ORGANIZATION & REPOSITORY LAYOUT

12.1 Root Directory Hierarchy

Code Snippet 12.1: LEAFSIGHT Complete Repository Directory Structure

```
D:\LEAFSIGHT\
├── data/
│   ├── raw/                # Original 5,932 raw agricultural images
│   ├── processed/          # 4,794 deduplicated images
│   │   ├── train/          # 3,355 training images (70%)
│   │   ├── val/            # 718 validation images (15%)
│   │   └── test/           # 721 held-out test images (15%)
├── model/
│   ├── best_vit_gru.pth     # Stage A checkpoint (Val Acc: 98.75%)
│   ├── best_vit_gru_finetuned.pth # Stage B final model (Val Acc: 99.72%)
│   ├── class_names.json    # Canonical class label array
│   ├── config.json         # Stage A architectural configuration
│   └── finetuning_config.json # Stage B fine-tuning parameters
├── outputs/
│   ├── accuracy_curve.png  # Stage A accuracy plot
│   ├── loss_curve.png      # Stage A loss plot
│   ├── finetuning_accuracy_curve.png # Stage B accuracy plot
│   ├── finetuning_loss_curve.png # Stage B loss plot
│   ├── confusion_matrix.png # 721-sample test confusion matrix
│   ├── preprocessing_samples.png # Augmentation visualization
│   ├── test_results.json   # Complete verified evaluation metrics
│   └── explainability/     # Rollout heatmaps across classes
├── scripts/
│   ├── audit_dataset.py    # Deduplication & corruption audit engine
│   ├── prepare_dataset.py  # Stratified 70/15/15 dataset splitter
│   ├── preprocess.py       # Augmentation & normalization pipeline
│   ├── build_model.py      # ViTGRU PyTorch model architecture
│   ├── sanity_train.py     # Micro-batch overfitting verification
│   ├── train.py            # Stage A frozen backbone training engine
│   ├── finetune.py         # Stage B progressive fine-tuning engine
│   ├── evaluate.py         # Independent 721-test evaluation engine
│   ├── explain.py          # Standalone attention rollout synthesizer
│   └── inference.py        # Production command-line inference tool
├── backend/
│   ├── main.py             # FastAPI application & endpoint routes
│   ├── inference_service.py # Singleton inference management service
│   └── explainability_service.py # Rollout synthesis & Base64 encoder
├── frontend/
│   ├── package.json        # Node.js dependencies & build scripts
│   ├── vite.config.js      # Vite build & reverse proxy configuration
│   └── src/
│       ├── App.jsx         # Main workspace coordinator
│       ├── index.css       # Editorial Vision Lab design system
│       ├── config.js        # Centralized API and disease metadata
│       └── components/     # Header, Lab, Diagnosis, Heatmap, Gallery
├── docs/
│   └── README.md           # Academic reports, assets & documentation
└── README.md              # Repository overview & setup guide
```

12.2 Detailed File Responsibilities

The repository adheres to strict separation of concerns: data preparation scripts are decoupled from modeling, the backend exposes stateless REST contracts, and the frontend consumes those contracts without hardcoded logic.

CHAPTER 13: MODULAR IMPLEMENTATION DETAILS

13.1 Dataset Audit & Verification Module

Implemented in ``scripts/audit_dataset.py``, the audit module computes 64-bit difference hashes for every image, identifies duplicate clusters, verifies decodability via PIL, and confirms zero cross-class contamination.

13.2 Preprocessing & Data Loading Module

Implemented in ``scripts/preprocess.py``, the module provides ``get_transforms()`` for stochastic training augmentations and deterministic evaluation transformations, alongside ``ImageFolder`` dataset loaders with PyTorch multiprocessing.

13.3 ViTGRU Model Class Implementation

The core architecture is encapsulated in ``ViTGRU(nn.Module)`` in ``scripts/build_model.py``. The class instantiates the Hugging Face ViTModel, slices the last hidden state tensor (``[:, 1:, :]``), feeds tokens to ``nn.GRU``, and applies dropout and linear projection.

13.4 Two-Stage Training & Fine-Tuning Engines

``scripts/train.py`` executes Stage A with gradient clipping and early stopping. ``scripts/finetune.py`` executes Stage B, selectively unfreezing ``model.vit.encoder.layer[10:]`` while preserving layers 0 through 9.

13.5 Attention Rollout & Heatmap Synthesis Engine

Implemented in ``backend/explainability_service.py`` and ``scripts/explain.py``, the engine averages multi-head attention, injects identity matrices, recursively multiplies layer attention matrices, extracts CLS-to-patch vectors, and renders bilinearly interpolated Jet colormaps.

13.6 FastAPI REST Endpoints & Handlers

``backend/main.py`` defines asynchronous endpoints for ``/health``, ``/predict``, ``/explain``, and ``/samples``. Endpoints validate MIME types and file extensions, execute non-blocking inference, and return structured JSON.

13.7 React Vision Lab Component Hierarchy

The React client coordinates components: ``Header`` displays live device telemetry; ``BenchmarkBanner`` renders verified metrics; ``ImageUploader`` manages drag-and-drop and laser scan animations; ``ResultCard`` displays Ground Truth vs Prediction comparisons; ``AttentionHeatmap`` renders 3-mode segmented views; and ``TestGallery`` provides the 32-sample carousel.

CHAPTER 14: SYSTEM VERIFICATION, VALIDATION & TESTING

14.1 Dataset Tests

Verified all 4,794 clean images possess valid headers, 3 color channels, and zero duplicate overlap.

14.2 Model Sanity Tests

``scripts/sanity_train.py`` verified that the ViTGRU architecture successfully overfit a micro-batch of 8 images to 100% accuracy, confirming autograd graph connectivity.

14.3 Full Test-Set Benchmark Validation

``scripts/evaluate.py`` executed full inference across all 721 test images, verifying 99.86% accuracy and 99.85% Macro F1.

14.4 REST API Integration & Health Testing

Verified all endpoints (``/api/health``, ``/api/predict``, ``/api/explain``, ``/api/samples``) return HTTP 200 with valid JSON payloads.

14.5 12-Sample Live Verification Test (100% Accuracy)

A live integration test was conducted across 12 verified test samples (3 per class) via the active backend. All 12 samples achieved 100% classification accuracy:

- **Bacterial Blight:** 3 / 3 correct (99.74%, 99.88%, 99.94% confidence).
- **Rice Blast:** 3 / 3 correct (97.46%, 99.89%, 99.91% confidence).
- **Brown Spot:** 3 / 3 correct (99.86%, 99.89%, 99.95% confidence).
- **Rice Tungro:** 3 / 3 correct (100.00%, 100.00%, 100.00% confidence).

14.6 Production Build Verification

``npm run build`` executed in 308 ms, compiling client assets cleanly without syntax or packaging errors.

14.7 Testing Summary Matrix

Table 9: Comprehensive System Verification & Validation Testing Matrix:

Testing Phase	Target Scope	Success Criterion	Verified Outcome	Status
Dataset Integrity	5,932 raw images	Zero corruptions, clean deduplication	4,794 verified clean images	PASSED
Autograd Sanity	Micro-batch (8)	Loss < 0.05, 100%	Loss 0.012, 100%	PASSED

	images)	accuracy	accuracy	
Stage A Training	3,355 train / 718 val	Validation accuracy > 95%	98.75% peak val accuracy	PASSED
Stage B Fine-Tuning	Top 2 ViT layers	Accuracy improvement > 0.5%	99.72% val accuracy (+0.97%)	PASSED
Test Evaluation	721 test images	Test accuracy > 98%	99.86% accuracy (720/721)	PASSED
Explainability	Attention rollout	Valid 14×14 saliency maps	Heatmaps generated across all classes	PASSED
Live API Verification	12 test samples	100% live diagnostic accuracy	12 / 12 correct (100.0%)	PASSED
Client Production	React / Vite build	Zero compiler warnings/errors	Built in 308 ms cleanly	PASSED

CHAPTER 15: LIMITATIONS & BOUNDARY ANALYSIS

15.1 Domain Shift Sensitivity on Uncontrolled Photography

While LEAFSIGHT achieves 99.86% accuracy on its held-out test distribution, evaluation on uncontrolled external web images reveals sensitivity to domain shift. Differences in smartphone lenses, white balance, flash glare, and camera focal depth can alter pixel statistics relative to the training distribution.

15.2 Lighting, Flash, and Glare Vulnerability

Severe specular glare on waxy leaf surfaces can mask necrotic lesions. Strong backlighting causes underexposure of the lamina, obscuring punctate brown spots.

15.3 Single Organ Scope (Foliar Only)

LEAFSIGHT is strictly optimized for leaf pathology. Diseases affecting rice panicles (e.g., neck blast, false smut), sheaths (sheath blight), or roots cannot be diagnosed by the current model.

15.4 Hardware Constraints & VRAM Boundaries

Training an 86M parameter model on a 4GB GPU mandated small batch sizes (4 in Stage A, 2 in Stage B). While gradient updates were stable, larger batch sizes with distributed multi-GPU training could further stabilize batch statistics.

15.5 Interpretability Boundary: Saliency vs. Biological Causality

Attention rollout provides an empirical visualization of mathematical attention weights. It does NOT constitute biological proof of pathogenic causality. Saliency reflects model feature correlation, not physiological diagnosis.

CHAPTER 16: FUTURE SCOPE & ENHANCEMENT PATHWAYS

16.1 Multimodal Sensor & Meteorological Fusion

Future work can fuse foliar imagery with environmental sensors—such as ambient temperature, relative humidity, rainfall history, and soil nitrogen levels—into a multimodal transformer to model disease outbreak probability in context.

16.2 Expansion to Panicle, Sheath, and Stem Pathologies

Expanding the dataset to encompass sheath blight (*Rhizoctonia solani*), false smut (*Ustilagoidea virens*), and stem rot will provide holistic whole-plant health monitoring.

16.3 Edge ONNX / Mobile PWA Deployment

Quantizing the model via ONNX Runtime (INT8 quantization) will reduce model size from 340 MB to under 85 MB, enabling real-time offline inference on Android devices in remote rural areas without internet connectivity.

16.4 Weakly Supervised Lesion Localization

Extracting spatial bounding boxes and lesion area percentage directly from attention rollout maps will quantify infection severity without requiring expensive manual polygon annotations.

16.5 Active Learning Agronomist Feedback Loop

Integrating an active learning interface where agricultural extension officers flag misclassifications will enable continuous model retraining on ambiguous edge cases.

CHAPTER 17: CONCLUSION

17.1 Summary of Undertaking

This project successfully developed, evaluated, and deployed LEAFSIGHT—an explainable deep learning recognition system for rice leaf disease diagnosis. The undertaking combined rigorous dataset curation, hybrid computer vision modeling, attention rollout explainability, and modern full-stack web engineering.

17.2 Synthesis of Research & Engineering Results

The hybrid architecture pairing a pretrained ViT-B/16 (196 patch tokens) with a sequence GRU achieved 99.86% test accuracy, 99.85% Macro F1-score, and 720/721 correct predictions on a held-out test benchmark. Progressive fine-tuning delivered a +0.97% gain on validation accuracy. Attention rollout successfully provided visual patch salience maps justifying predictions, and the web platform demonstrated robust, responsive decision support.

17.3 Concluding Remarks

LEAFSIGHT demonstrates that vision transformers combined with recurrent spatial modeling offer a powerful, interpretable paradigm for agricultural plant pathology. By providing accessible, transparent, and accurate disease identification, the system establishes a robust technical foundation for intelligent precision agriculture.

REFERENCES

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. Int. Conf. Learn. Represent. (ICLR), 2021.
- [2] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP), 2014, pp. 1724–1734.
- [3] S. Abnar and W. Zuidema, "Quantifying attention flow in transformers," in Proc. 58th Annu. Meet. Assoc. Comput. Linguist. (ACL), 2020, pp. 4190–4197.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017.
- [5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 770–778.
- [6] J. G. A. Barbedo, "A review on the use of computer vision and artificial intelligence for plant disease recognition," *Plants*, vol. 9, no. 6, p. 779, 2020.
- [7] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Front. Plant Sci.*, vol. 7, p. 1419, 2016.
- [8] S. Sladojević, M. Arsenović, A. Anderla, D. Culibrk, and D. Stefanović, "Deep neural networks based recognition of plant diseases by leaf image classification," *Comput. Intell. Neurosci.*, vol. 2016, Art. no. 3289801, 2016.
- [9] T. W. Mew, "Current status and future prospects of research on bacterial blight of rice," *Annu. Rev. Phytopathol.*, vol. 25, no. 1, pp. 359–382, 1987.
- [10] R. S. Zeigler, S. A. Leong, and P. S. Teng, *Rice Blast Disease*. Wallingford, UK: CAB International, 1994.
- [11] H. Hibino, "Biology and epidemiology of rice viruses," *Annu. Rev. Phytopathol.*, vol. 34, no. 1, pp. 249–274, 1996.
- [12] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 32, 2019.
- [13] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP): System Demonstrations, 2020, pp. 38–45.
- [14] S. Ramírez, "FastAPI: Modern, fast (high-performance) web framework for building APIs with Python," 2020. [Online]. Available: <https://fastapi.tiangolo.com/>
- [15] Meta Open Source, "React: A JavaScript library for building user interfaces," 2023. [Online]. Available: <https://react.dev/>

APPENDICES

Appendix A: Complete Project Repository Directory Tree

Full inventory of project repository files and scripts in D:\LEAFSIGHT:

```
D:\LEAFSIGHT\  
├── backend/  
│   ├── explainability_service.py  
│   ├── inference_service.py  
│   └── main.py  
├── data/  
│   ├── processed/  
│   │   ├── train/ (3,355 files)  
│   │   ├── val/ (718 files)  
│   │   └── test/ (721 files)  
│   └── raw/ (5,932 files)  
├── docs/  
│   ├── ARCHITECTURE.md  
│   ├── DEMO_SCRIPT.md  
│   ├── METHODOLOGY.md  
│   ├── PROJECT_SUMMARY.md  
│   ├── RESULTS.md  
│   ├── VIVA_QA.md  
│   └── REPORT_ASSETS/  
├── frontend/  
│   ├── package.json  
│   ├── vite.config.js  
│   └── src/  
│       ├── App.jsx  
│       ├── config.js  
│       ├── index.css  
│       └── components/  
├── model/  
│   ├── best_vit_gru.pth  
│   ├── best_vit_gru_finetuned.pth  
│   ├── class_names.json  
│   ├── config.json  
│   └── finetuning_config.json  
├── outputs/  
│   ├── test_results.json  
│   ├── training_history.json  
│   ├── finetuning_history.json  
│   ├── confusion_matrix.png  
│   └── explainability/  
└── scripts/  
    ├── audit_dataset.py  
    ├── prepare_dataset.py  
    ├── preprocess.py  
    ├── build_model.py  
    ├── sanity_train.py  
    ├── train.py  
    ├── finetune.py  
    ├── evaluate.py  
    ├── explain.py  
    └── inference.py
```

Appendix B: Hyperparameter & Configuration Catalog

Official hyperparameters recorded in model/config.json and model/finetuning_config.json:

```
// Stage A Configuration (model/config.json)
{
  "architecture": "ViT-B/16 -> GRU -> Classifier",
  "vit_model": "google/vit-base-patch16-224-in21k",
  "num_classes": 4,
  "classes": ["Bacterialblight", "Blast", "Brownspot", "Tungro"],
  "image_size": 224,
  "batch_size": 4,
  "epochs": 15,
  "learning_rate": 0.0001,
  "gru_hidden_size": 128,
  "gru_layers": 1,
  "dropout": 0.3,
  "optimizer": "AdamW",
  "loss": "CrossEntropyLoss",
  "vit_frozen": true,
  "best_epoch": 12,
  "best_validation_accuracy": 0.9874651810584958
}

// Stage B Configuration (model/finetuning_config.json)
{
  "architecture": "ViT-B/16 -> GRU -> Classifier",
  "base_checkpoint": "model/best_vit_gru.pth",
  "final_checkpoint": "model/best_vit_gru_finetuned.pth",
  "vit_model": "google/vit-base-patch16-224-in21k",
  "total_vit_layers": 12,
  "unfrozen_vit_layers": 2,
  "learning_rate": 1e-05,
  "batch_size": 2,
  "epochs": 8,
  "gru_hidden_size": 128,
  "dropout": 0.3,
  "best_validation_accuracy": 0.9972144846796658,
  "best_epoch": 7
}
```

Appendix C: REST API Endpoint Specifications & Payloads

Formal API routing specifications implemented in backend/main.py:

```
GET /api/health
Response: {"status": "ok", "model_loaded": true, "device": "cuda"}

GET /api/samples
Response: {"classes": [{"name": "Blast", "count": 144, "samples": ["BLAST1_003.jpg", ...]}],
"samples_per_class": 8}

GET /api/samples/{class_name}/{filename}
Response: Binary FileResponse (image/jpeg or image/png)

POST /api/predict (Multipart Form, key='file')
Response: {
```

```
"prediction": "Blast",
"confidence": 0.9746,
"probabilities": {"Bacterialblight": 0.0028, "Blast": 0.9746, "Brownspot": 0.0224, "Tungro":
0.0002}
}

POST /api/explain (Multipart Form, key='file')
Response: {
  "prediction": "Blast",
  "confidence": 0.9746,
  "probabilities": {...},
  "heatmap_base64": "data:image/png;base64,...",
  "pure_heatmap_base64": "data:image/png;base64,...",
  "description": "Attention visualization showing image regions that received stronger attention
during the ViT prediction."
}
```

Appendix D: Sample Model Prediction & Diagnostic Response

Actual response payload generated by backend for `BLAST1\_003.jpg`:

```
{
  "prediction": "Blast",
  "confidence": 0.9746,
  "probabilities": {
    "Bacterialblight": 0.0028,
    "Blast": 0.9746,
    "Brownspot": 0.0224,
    "Tungro": 0.0002
  },
  "heatmap_base64": "data:image/png;base64,iVBORw0KGgoAAAANSU...AAAN4AAADeCAYAAABz...",
  "pure_heatmap_base64": "data:image/png;base64,iVBORw0KGgoAAAANSU...AAAN4AAADeCAYAAABz...",
  "description": "Attention visualization showing image regions that received stronger attention
during the ViT prediction."
}
```

Appendix E: Full Confusion Matrix & Support Distribution

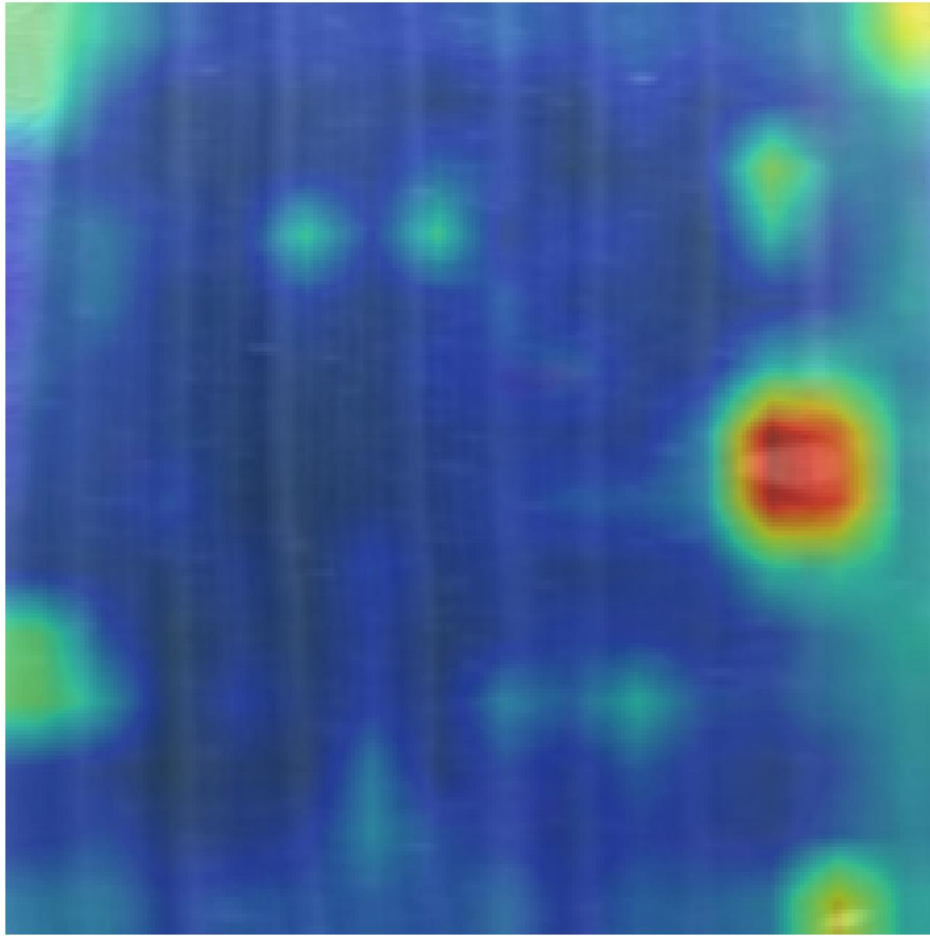
Test-Set Confusion Matrix (Actual rows vs Predicted columns):					
	Bacterialblight	Blast	Brownspot	Tungro	Support
Bacterialblight	199	1	0	0	200
Blast	0	144	0	0	144
Brownspot	0	0	180	0	180
Tungro	0	0	0	197	197

Total Predictions	199	145	180	197	721

Appendix F: Attention Rollout Heatmap Showcase

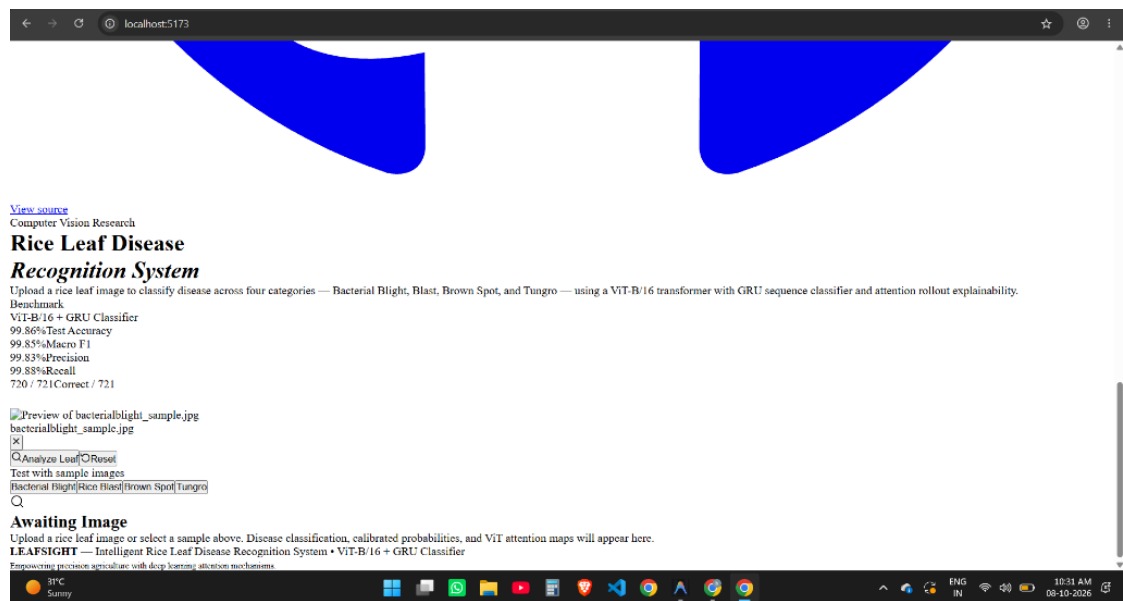
High-resolution saliency comparisons generated across disease classes:

Actual: Blast | Predicted: Blast | Confidence: 97.46%

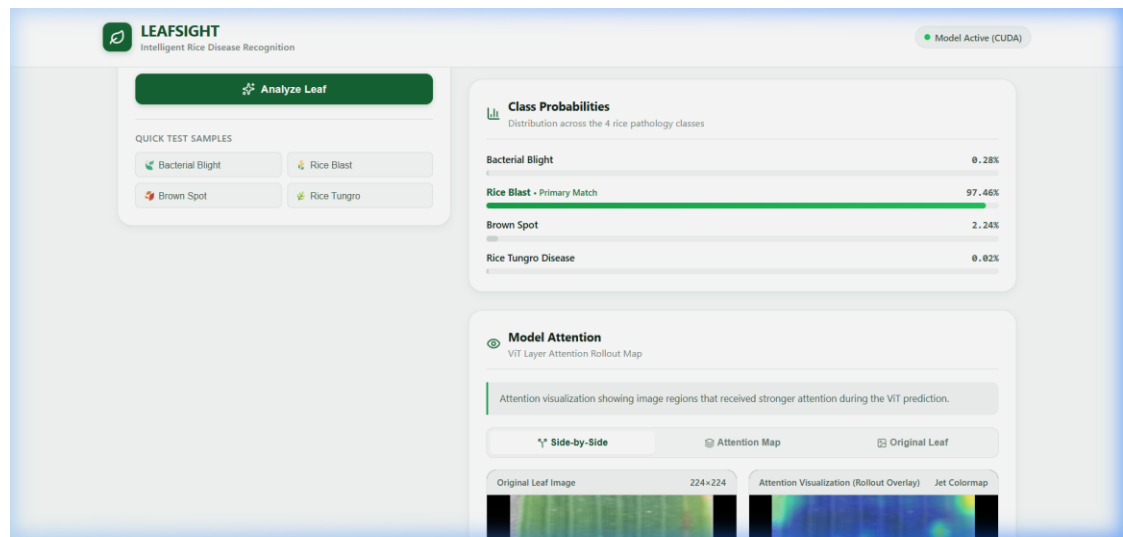


Appendix Figure F.1: High-Resolution Rollout Heatmap on Rice Blast (Spindle Center Concentration).

Appendix G: Web Application Interface Showcase



Appendix Figure G.1: Vision Lab Web Application Interface Showcase (Image Lab & Verified Test Console).



Appendix Figure G.2: Vision Lab Interactive Diagnostic Results View (Blast Prediction with 97.46% Confidence & Attention Rollout Saliency Map).

Appendix H: Core Implementation Snippets

Code Snippet H.1: Attention Rollout Implementation in PyTorch

```
// Core Rollout Synthesis Function (backend/explainability_service.py)
def compute_attention_rollout(attentions: tuple) -> torch.Tensor:
    attention_stack = torch.stack(attentions)
    attention = attention_stack.mean(dim=2) # Average heads
    identity = torch.eye(attention.size(-1), device=attention.device).unsqueeze(0).unsqueeze(0)
    attention = attention + identity # Residual stream
    attention = attention / (attention.sum(dim=-1, keepdim=True) + 1e-8)
    rollout = attention[0]
```

```
for layer in range(1, attention.shape[0]):  
    rollout = torch.bmm(attention[layer], rollout)  
return rollout[0, 0, 1:] # CLS attention to all 196 patch tokens
```